M.Sc. Engg. (CSE) Thesis — Appendices

Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models

Supplementary Appendices

Companion volume to the thesis of the same title

Submitted by

Md. Sakif Khan

0421052099

Supervised by

Dr. Sadia Sharmin

Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

August 2026

Contents

List of Tables

List of Algorithms

About This Volume

This volume holds the five appendices of the thesis named on the title page. They were bound separately so that the thesis itself could be read at the length its argument needs, and nothing that a sceptical reader might want to check has been dropped to achieve that: the prompt texts, the tool specification, the question samples and run manifests, the annotation instructions and label schema, and the implementation and measurement notes are all reproduced here in full.

The two volumes cross-reference each other, and their numbering is what tells them apart. *Everything in this volume is lettered*: Appendix B is a chapter of it, Appendix E.3 a section, Table E.2 a table, Algorithm E.1 an algorithm. *Everything in the main volume is numbered*: Chapter 4, Section 5.3, Table 7.2, Algorithm 1. So a reference naming a letter points here and a reference naming a digit points into the thesis, wherever it appears and whatever it names. A reference that crosses between the volumes is not a live hyperlink, since its destination is in the other file; one that stays within a volume is.

Both volumes are generated from one source tree, and every number either of them states about the other is read from that build rather than typed in. A reference that prints as `??` means the volumes were built out of step, not that the target is missing.

Appendix A

Prompt Templates

Seven prompts constitute the whole of AGR's interface to the language model. Six more serve the baselines: two shared by the three static baselines, and four belonging to the reimplemented agentic baseline. All thirteen are reproduced here verbatim from the source that sends them, in the order a question encounters them. The agentic baseline's four matter more to a sceptical reader than any of AGR's own, because Section 5.4.3 rests the central comparison on a reimplementation rather than on the authors' code, and these prompts are what that reimplementation actually is. Braces written `{like this}` are Python format fields; doubled braces `{{...}}` are literal braces in the emitted text.

This appendix carries a checker rather than a promise. scripts/check_appendix_prompts.py extracts each constant from the source and asserts that it still appears here. Nothing in the LaTeX build could detect the divergence on its own, and the reproducibility argument of Section 5.6.2 depends on the two not diverging: the response cache keys on prompt text, so a prompt edited in one place and not the other invalidates every cached run as well as this appendix.

Two properties of this set matter more than any individual wording. First, no template carries a configuration constant: neither α nor τ reaches the model in any prompt, which is what allows one cached response to be shared across every condition of the α–τ sweep (Section 5.7.1). Second, budgets are counters in code (Section 4.7) rather than instructions: no prompt tells the model to stop, and stopping does not depend on its cooperation. One template qualifies that second property without breaking it — the evaluator of Appendix A.3 is passed the remaining depth and backtracks as decision context, the only budget figures any prompt carries, substituted at send time rather than written into the template, and the note there records why a model that ignores them changes nothing.

A.1 Planner

Sent once per question, unless the planner is ablated. Section 4.3 describes the four-archetype few-shot design and the topic-mention restriction; this is the template that implements both. The instruction to use as few sub-objectives as possible is the provision Section 5.15.1 finds to have been insufficient on the shallower benchmark.

```
You decompose questions into a minimal ordered chain of sub-objectives for
↪
navigating a knowledge graph. Each sub-objective must be answerable by
following one relation (possibly through an event node). Use #N to
↪ reference
the result of sub-objective N. Use as FEW sub-objectives as possible.

For topic_mentions, extract ONLY specific named entities that appear
literally in the question (people, places, organizations, works, products)
↪ .
Do NOT extract generic type words such as "celebrity", "university",
"country", "senator", "airport", or "currency" -- these describe what
↪ kind
of answer is wanted, not where to start searching.

Example 1 (single hop -- do NOT decompose):
Question: "who is the president of France?"
{"sub_objectives": ["find the president of France"],
 "topic_mentions": ["France"]}

Example 2 (composition):
Question: "who directed the film that won Best Picture in 1998?"
{"sub_objectives": ["find the film that won Best Picture in 1998",
                    "find the director of #1"],
 "topic_mentions": ["Best Picture"]}

Example 3 (conjunction):
Question: "which countries border France and have Spanish as official
↪ language?"
{"sub_objectives": ["find countries that border France",
                    "find which of #1 have Spanish as an official
↪ language"],
 "topic_mentions": ["France", "Spanish"]}

Example 4 (superlative):
Question: "what is the largest city in the country where the Nile ends?"
{"sub_objectives": ["find the country where the Nile ends",
                    "find the largest city in #1"],
 "topic_mentions": ["Nile"]}
```

```
Question: "{question}"
```

The plan this returns is validated rather than trusted. Four checks run before it is accepted, and any failure installs the single-objective fallback described in Section 4.3: the sub-objective list must be non-empty, it must hold no more than four entries, every `#N` reference must point strictly backwards, and at least one topic mention must be present.

A.2 Relation Scorer

Sent once per expansion step, scoring all candidate edges across all anchors in a single call. This is the prompt that carries the property discussed in Section 4.4.2: it mentions neither α nor the anchor ordering, so its response is reusable across every sweep condition.

```
Sub-objective: {objective}

Candidate knowledge-graph edges (anchor entity --relation-->):
{candidates}

For EACH candidate, rate from 0.0 to 1.0 how likely following that edge
↪ from
that anchor leads toward completing the sub-objective. 1.0 = directly
answers it, 0.0 = irrelevant. Judge the RELATION MEANING in context of
↪ the
anchor, not surface word overlap.
```

The final instruction exists because of the failure it names. Judging surface word overlap is what an embedding does, and the blend of Section 4.4.2 already supplies that signal; the model is asked for the part the embedding cannot give.

A.3 Evaluator

Sent once per navigation cycle. The fact window is the thirty triples ranked most relevant to the current sub-objective, not the thirty most recent — the change forced by the failure recorded in Table E.1.

```
Current sub-objective: {objective}
Recently retrieved facts:
{facts}
Remaining budget: {d_left} more expansions, {b_left} more backtracks.

Decide:
```

```
- "answer" if the facts complete the sub-objective (list satisfying
↪ entities
in "resolved", exactly as named in the facts),
- "continue" if this direction is promising but incomplete,
- "backtrack" if this direction is a dead end.
```

The remaining budget is stated for context, not for enforcement. A model that ignores it changes nothing: the counters in Section 4.7 terminate the run regardless of what the evaluator decides.

A.4 Drafter and Claim Decomposer

One call produces the draft answer, its asserted entity list, and the atomic claims the verification layer will check. Section 4.11 explains why these are one call rather than three: a draft and a claim set produced separately can disagree about what was asserted, and the layer would then verify something the reader never sees.

Four provisions here are load-bearing, and Section 4.11 gives the case that forced each. Copying entity names exactly is what keeps the graph's spelling — University Yale rather than Yale University — in the answer. The distinction between entities the draft *asserts* and entities it merely mentions is what stops a hedge from being scored as an assertion. The anti-vacuity instruction is what stops the model answering "a university" to a *which university* question. And requiring a hedge to carry zero claims is what makes an abstention verifiable rather than merely unverified.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}
Entities identified as promising during exploration:
{candidates}

1. Draft an answer using ONLY the facts and candidates above. Use entity
   names exactly as written. If insufficient, say what could not be
   determined.
2. List the answer entities themselves in "answer_entities", copied
↪ EXACTLY
   as they are named in the facts above -- these are the entities your
↪ draft
   asserts as the answer, not every entity it mentions. If your draft is
↪ a
   hedge ("could not be determined"), "answer_entities" MUST be [].
   IMPORTANT: "answer_entities" must be specific entities of the kind the
   question asks for (a person for "who", a place for "where", etc.).
   Never restate the question or name entity types/categories as the
   answer. If you cannot name specific answer entities from the facts,
```

```
   your draft must be a hedge and "answer_entities" must be [].
3. Decompose your draft into atomic factual claims, each as
   {{"h": entity name, "r": short relation phrase, "t": entity name}}.
   Only include claims your draft actually asserts. A hedged draft has
↪ zero
   claims.
```

A.5 Entailment Check

Sent only for claims that the two structural routes of Section 4.12.1 could not settle. It is the semantic fallback, and the definition of *unsupported* is given explicitly because the default reading of the word is too permissive: a model asked merely whether a claim is supported will accept anything it believes to be true.

```
Retrieved knowledge-graph triples:
{facts}

For each claim below, answer whether it is directly supported by the
↪ triples
above (true) or not (false). Unsupported includes: plausible but absent,
contradicted, or only inferable via outside knowledge.
Claims:
{claims}
```

The three enumerated cases are the three ways a fluent model fails this task. *Plausible but absent* is the hallucination the layer exists to catch. *Contradicted* is rarer and easier. *Only inferable via outside knowledge* is the one that matters most for the thesis's claim, because a model drawing on parametric memory here would certify exactly the content the graph was supposed to constrain.

A.6 Rewriter

Sent when verification rejects at least one claim but at least one entity survives. It is the mechanism by which an unsupported claim is removed rather than merely flagged.

```
Question: {question}
Draft answer: {draft}
The following claims in the draft are NOT supported by the knowledge
↪ graph:
{unsupported}
The following entities remain verified and MUST be kept in the answer,
named exactly as written: {kept}

Rewrite the draft: keep the verified entities and their supported claims,
```

```
and remove or explicitly hedge the unsupported claims. If the kept list
↪ is
empty, state that the answer could not be verified against the knowledge
graph. Respond with prose only.
```

The kept list is computed in code by the deterministic filter of Section 4.13.4, not proposed by the model. The rewrite call may reword the answer but cannot change which entities it asserts, and Section 4.14.1 reads the case in which that separation decided whether a question was scored correct.

A.7 Answerer

Sent on the path where verification is skipped, which is the draft-only ablation of Section 5.12, and on the give-up path where the budget drained before a draft existed.

```
Question: {question}
Facts retrieved from the knowledge graph:
{facts}

Entities identified as promising during exploration:
{candidates}

Answer using ONLY these facts and candidates. Give the answer entity name
↪ (s)
exactly as they appear above. If the information is insufficient, state
↪ what
could not be determined.
Also list in "answer_entities" the entities your answer
asserts (exactly as named above); [] if it asserts nothing.
```

Comparing this against the drafter of Appendix A.4 shows what the draft-only ablation removes. The anti-vacuity provision is absent here, and Section 5.12 reports the hedge-rate consequence.

A.8 Baseline Prompts

The baselines share the backbone, the budget, and the answer-entity contract of the full system, so that the comparison isolates retrieval rather than output format. All three emit `{"answer": "...", "answer_entities": ["..."]}`.

No-retrieval

The parametric control. It receives the question and nothing else, which is what makes it a measurement of memorisation rather than of retrieval (Section 5.4).

```
Answer this question with the name(s) of the answer entity or
entities. Be concise. If you do not know, say so and return [].

Question: {question}
```

Vector-RAG and GraphRAG

Both use the template below; GraphRAG imports it from the Vector-RAG module precisely so that the two differ only in how `{facts}` was retrieved. Section 5.4 records the consequence noted in Section 5.4.4: this prompt is more abstinent than the full system's drafter, and the resulting hedge rates are a property of the prompt as much as of the retrieval.

```
Facts (retrieved from a knowledge graph):
{facts}

Question: {question}
Answer using ONLY the facts above. Name entities exactly as written in
↪ the
facts. If the facts are insufficient, say so and return [].
```

A.9 Agentic Baseline (Think-on-Graph)

Four prompts implement the reimplementation of Section 5.4.3, at beam width 3 and depth 3. They are given in full because the thesis's central comparison is against this code rather than against published numbers (Section 5.8), and a reader who wants to challenge that comparison should be able to read what was actually run. The structure follows the original: prune relations, expand, prune entities, ask whether what has been gathered suffices, and answer if it does.

The contrast with AGR's scorer is worth noting while reading the first of them. This prompt *selects* a set of relations; AGR's scores every candidate on $[0, 1]$ and blends that with an embedding similarity (Section 4.4.2). Selection cannot express "all of these are poor", which is what the low-signal backtrack trigger needs to fire on.

Relation pruning

```
Question: {question}
Current entity: {entity}
```

```
Available relations (relation, direction, fanout):
{relations}
Select up to {w} relations most likely to lead to the answer.
Return {{"relations": [{{"rel": "...", "dir": "in|out"}}]}}.
```

Entity pruning

```
Question: {question}
Candidate entities reached (via {entity} --{rel}-->):
{entities}
Select up to {w} entities most likely on the path to the answer.
Return {{"entities": ["..."]}}.
```

Sufficiency check

This is the prompt the call budget most often interrupts. It runs once per depth, and a run clipped before it returns true answers from whatever the beam has gathered — the mechanism behind the clipped-subset accuracies of Table 5.6.

```
Question: {question}
Knowledge triples gathered so far:
{triples}
Can the question be answered from these triples alone?
Return {{"sufficient": true|false}}.
```

Answer

```
Question: {question}
Knowledge triples:
{triples}
Answer using ONLY these triples; name entities exactly as written.
If insufficient, say so and return [].
```

Appendix B

Tool API Specification and Cypher Templates

Section 4.2 argues that the model must never author a query. This appendix gives the five operations that follow from that decision, with the Cypher each one executes. Every query below is a fixed template. The model supplies values for the $-parameters and nothing else, so a malformed model output produces a failed lookup rather than a syntax error, and the error taxonomy of Section 5.13 never has to separate reasoning failures from query-authoring failures.

Two limits appear throughout. max_relations, set to 300, caps the relation list returned for an entity; max_fanout, set to 200, caps the neighbours returned for one edge. Table E.1 records why the first was raised from its original value of 80.

Every invocation is appended to a JSONL log with its arguments, its result, and its wall-clock duration in milliseconds. That log is the per-call record the latency figures of Section 5.11 are computed from, and for the determinism probes it is the only surviving per-question artifact.

B.1 search_entity

Resolves a surface form to graph entities. This is the only entry point from text to graph, and the only operation that consults an embedding. It delegates to the three-stage resolver of Section 3.2.6 — exact, lexical, vector — and returns which stage produced the hit, so that a downstream failure can be attributed to resolution rather than to navigation.

One naming collision has to be declared rather than tidied away. The returned JSON key is literally tier, and it takes the values 1, 2 and 3 for the three stages. Section 5.5.3 reserves that word for the groundedness metric, and this is the sole exception: the key appears verbatim in every committed tool log and in the scripts that read them, so renaming it in this appendix would misdocument the artifacts. The two senses are unrelated — a resolver stage is not a groundedness tier — and the stage names are used everywhere the prose has a choice.

```
search_entity(surface_form, k=5)
  -> [{id, name, score, tier}, ...]     up to k candidates

tier=1  exact:     MATCH (e:Entity {name:$m}) RETURN e.id, e.name
tier=2  lexical:   CALL db.index.fulltext.queryNodes('entity_name', $q)
                     YIELD node, score
                   RETURN node.id, node.name, score LIMIT $k
tier=3  vector:    CALL db.index.vector.queryNodes('entity_embedding', $k,
↪  $v)
                     YIELD node, score
                   RETURN node.id, node.name, score
```

The lexical stage accepts its top hit only when the full-text score exceeds a threshold of 1.0. Without that gate the index returns a ranked list for almost any input, including inputs with no correct answer in the graph, and the vector stage would never be reached.

B.2 get_relations

Returns the relation types incident to an entity, in *both* directions, each with its fanout count. Direction is carried explicitly because Freebase edges are asserted in one direction only, and the answer to a question is as often reached by traversing an edge backwards as forwards.

```
get_relations(entity_id)
  -> [{rel, dir, n}, ...]     sorted by n descending, capped at
↪ max_relations

MATCH (e:Entity {id:$id})-[r]->()
RETURN r.fb_name AS rel, 'out' AS dir, count(*) AS n
UNION ALL
MATCH (e:Entity {id:$id})<-[r]-()
RETURN r.fb_name AS rel, 'in' AS dir, count(*) AS n
```

Administrative predicates are filtered before the cap is applied, by prefix: common.topic., common., freebase., type., kg., user., dataworld., rdf-schema# and owl#. These are schema bookkeeping rather than facts about the world. The list is quoted as the code writes it, which is why it holds nine entries where Section 4.2 names eight: common.topic. is subsumed by common. and filters nothing the shorter prefix does not already filter. They also have very high fanout, so filtering before sorting is what keeps them from consuming the whole list.

B.3 get_neighbors

Expands one edge from one entity. This is the operation that makes Freebase's mediator nodes invisible to the agent: where a neighbour is a mediator, the tool hops through it in the same call

and returns the entity on the far side, carrying the via chain that reached it.

```
get_neighbors(entity_id, relation, direction='out')
  -> {neighbors: [{id, name, via[, cvt]}, ...], truncated: bool}

MATCH (e:Entity {id:$id})
MATCH (e)-[r {fb_name:$rel}]->(n)        -- or <-[r]- when direction='in'
RETURN n.id, n.name, n.is_cvt LIMIT $cap

-- for each neighbour with is_cvt = true, one further hop:
MATCH (c:Entity {id:$cid})-[r2]-(m:Entity)
WHERE m.id <> $orig AND m.is_cvt = false
RETURN m.id, m.name, r2.fb_name AS rel2 LIMIT $cap
```

The via chain is what makes the traversal auditable. A two-element via means the result was reached through a mediator, and the cvt field names the mediator that was crossed. Section 3.2 states the two conventions this creates: a mediator counts as two edges wherever the thesis measures distance in the graph (the reachability gate, the hop strata), and as one step against the agent's depth budget, because the agent should not be charged twice for an artifact of the encoding.

The truncated flag was intended to report whether the fanout cap was hit, so that a truncated expansion — a plausible cause of a later failure — could be distinguished from a relation that was simply never explored. It does not deliver that, for the two reasons Section 4.7 records: it is returned to the caller but never written to the tool log, and it is computed on the row count before mediator expansion while the list is truncated after it. Nothing in this thesis reads it.

B.4 verify_triple

Asks whether a specific relation holds between two specific entities. It was built as the relation-aware structural route of the verification layer, and it is *not called by any node in the final design*: free-text claim relations do not map onto Freebase predicates, so requiring the relation to match rejected true claims, and verify_connection replaced it. Section 4.2.2 gives that history and Section 4.12.1 gives the two routes the verifier ended up with. It is reproduced here because it remains part of the tool API, is exercised by the integration tests, and is the natural primitive for the relation-aware check Section 6.3 proposes.

```
verify_triple(head_id, relation, tail_id)
  -> {direct, via_cvt, supported}

MATCH (h:Entity {id:$h}), (t:Entity {id:$t})
RETURN
  EXISTS { MATCH (h)-[{fb_name:$rel}]-(t) } AS direct,
  EXISTS { MATCH (h)-[{fb_name:$rel}]-(c:Entity {is_cvt:true})-[]-(t) }
```

```
    AS via_cvt
```

Both patterns are undirected. A claim is supported if either holds. The via_cvt arm exists for the same reason as the mediator hop in get_neighbors: a fact stated through a mediator is still a fact, and a verifier that missed it would reject true claims about exactly the compositional questions the thesis is about.

B.5 verify_connection

Asks only whether two entities are adjacent, directly or through one mediator. The relation is not named.

```
verify_connection(a_id, b_id)
  -> {direct, via_cvt, connected}

MATCH (a:Entity {id:$a}), (b:Entity {id:$b})
RETURN EXISTS { MATCH (a)-[]-(b) } AS direct,
       EXISTS { MATCH (a)-[]-(c:Entity {is_cvt:true})-[]-(b) } AS via_cvt
```

This is the fifth operation, and the one not in the original design. Section 4.2 records that it was added when the verification layer of Section 4.10 needed a relation-agnostic adjacency test. Its permissiveness is deliberate and is also its weakness: Section 4.15 analyses wrongful acceptance as a direct consequence of asking whether two entities are connected at all rather than whether they are connected in the way the claim asserts.

Appendix C

Sampled Question IDs and Run Manifests

Every number this thesis reports is computed over the question sets listed here. Section 5.2 describes how the two test samples were drawn and certified; Table C.1 gives their sizes and strata, and the lists below record their contents, so that a reader can reproduce a reported figure over exactly the questions it was computed from.

The identifiers are the dataset's own. A CWQ identifier carries the WebQSP identifier it was derived from as its prefix, which is why some of them are long.

Table C.1: The three question sets. The test samples are stratified by hop count, the development set by question shape.

Set	n	Strata
test_webqsp	400	h1 256, h2 128, h3plus 4, unreachable 12
test_cwq	400	h1 137, h2 211, h3plus 49, unreachable 3
dev80	80	conjunction 13, cvt_heavy 10, one_hop 18, two_hop 32, unanswerable_env 4, unanswerable_fake 3

C.1 WebQSP test sample ($n = 400$)

Source: results/phase4/test_webqsp.json.

```
WebQTest-0 WebQTest-100 WebQTest-1003 WebQTest-1007 WebQTest-1010 WebQTest-1011 WebQTest-1014
WebQTest-1020 WebQTest-1022 WebQTest-103 WebQTest-1035 WebQTest-1047 WebQTest-105
WebQTest-1057 WebQTest-1059 WebQTest-1061 WebQTest-1063 WebQTest-1067 WebQTest-1069
WebQTest-1077 WebQTest-1079 WebQTest-108 WebQTest-1080 WebQTest-1090 WebQTest-1095
WebQTest-1097 WebQTest-1099 WebQTest-1108 WebQTest-1110 WebQTest-1111 WebQTest-1114
WebQTest-1115 WebQTest-1120 WebQTest-1121 WebQTest-1130 WebQTest-1133 WebQTest-114
WebQTest-1147 WebQTest-1151 WebQTest-1169 WebQTest-1170 WebQTest-1177 WebQTest-1178
WebQTest-118 WebQTest-1187 WebQTest-1189 WebQTest-1199 WebQTest-121 WebQTest-1215 WebQTest-122
WebQTest-1226 WebQTest-1233 WebQTest-1239 WebQTest-124 WebQTest-1242 WebQTest-1259
WebQTest-1267 WebQTest-1269 WebQTest-127 WebQTest-1271 WebQTest-128 WebQTest-1281
WebQTest-1289 WebQTest-129 WebQTest-1291 WebQTest-1292 WebQTest-1296 WebQTest-1304
WebQTest-1309 WebQTest-1313 WebQTest-132 WebQTest-1321 WebQTest-1322 WebQTest-1324
```

```
WebQTest-1327 WebQTest-133 WebQTest-1330 WebQTest-1334 WebQTest-1339 WebQTest-1350
WebQTest-1353 WebQTest-1354 WebQTest-1362 WebQTest-1367 WebQTest-1369 WebQTest-1370
WebQTest-1373 WebQTest-1376 WebQTest-1381 WebQTest-1387 WebQTest-1399 WebQTest-1416
WebQTest-1422 WebQTest-1426 WebQTest-1445 WebQTest-1452 WebQTest-1456 WebQTest-1457
WebQTest-1458 WebQTest-1461 WebQTest-1464 WebQTest-1465 WebQTest-147 WebQTest-1470
WebQTest-1471 WebQTest-1477 WebQTest-1484 WebQTest-1485 WebQTest-1490 WebQTest-1493
WebQTest-150 WebQTest-1501 WebQTest-1520 WebQTest-1524 WebQTest-153 WebQTest-1534
WebQTest-1548 WebQTest-1550 WebQTest-1555 WebQTest-1556 WebQTest-1557 WebQTest-1560
WebQTest-1561 WebQTest-1564 WebQTest-1566 WebQTest-1567 WebQTest-1572 WebQTest-1575
WebQTest-1584 WebQTest-1585 WebQTest-1595 WebQTest-16 WebQTest-161 WebQTest-1611 WebQTest-1618
WebQTest-1620 WebQTest-1628 WebQTest-1629 WebQTest-1630 WebQTest-1633 WebQTest-1635
WebQTest-164 WebQTest-1646 WebQTest-165 WebQTest-1650 WebQTest-1655 WebQTest-1658
WebQTest-1667 WebQTest-1668 WebQTest-1669 WebQTest-1673 WebQTest-1674 WebQTest-1679
WebQTest-1680 WebQTest-1682 WebQTest-1687 WebQTest-1688 WebQTest-1698 WebQTest-1707
WebQTest-1711 WebQTest-172 WebQTest-1723 WebQTest-1724 WebQTest-1725 WebQTest-1730
WebQTest-1733 WebQTest-1736 WebQTest-1737 WebQTest-1744 WebQTest-1752 WebQTest-1758
WebQTest-1759 WebQTest-176 WebQTest-1760 WebQTest-1763 WebQTest-1764 WebQTest-1767
WebQTest-1768 WebQTest-177 WebQTest-1770 WebQTest-1782 WebQTest-1791 WebQTest-1793
WebQTest-1794 WebQTest-1795 WebQTest-1796 WebQTest-1813 WebQTest-1822 WebQTest-1824
WebQTest-1829 WebQTest-1835 WebQTest-1836 WebQTest-1840 WebQTest-1847 WebQTest-1857
WebQTest-186 WebQTest-1864 WebQTest-1868 WebQTest-1876 WebQTest-1877 WebQTest-1897
WebQTest-1898 WebQTest-1902 WebQTest-1907 WebQTest-1909 WebQTest-191 WebQTest-1914
WebQTest-1915 WebQTest-1919 WebQTest-1920 WebQTest-1923 WebQTest-1928 WebQTest-1929
WebQTest-193 WebQTest-1937 WebQTest-194 WebQTest-1940 WebQTest-1942 WebQTest-195 WebQTest-1954
WebQTest-1962 WebQTest-1967 WebQTest-1968 WebQTest-1969 WebQTest-1971 WebQTest-1972
WebQTest-1976 WebQTest-1978 WebQTest-1979 WebQTest-1981 WebQTest-1985 WebQTest-1993
WebQTest-200 WebQTest-2002 WebQTest-2003 WebQTest-2006 WebQTest-2017 WebQTest-2020
WebQTest-205 WebQTest-206 WebQTest-209 WebQTest-211 WebQTest-215 WebQTest-225 WebQTest-23
WebQTest-231 WebQTest-233 WebQTest-234 WebQTest-239 WebQTest-245 WebQTest-254 WebQTest-256
WebQTest-262 WebQTest-264 WebQTest-273 WebQTest-284 WebQTest-286 WebQTest-291 WebQTest-296
WebQTest-298 WebQTest-3 WebQTest-301 WebQTest-304 WebQTest-308 WebQTest-310 WebQTest-311
WebQTest-314 WebQTest-316 WebQTest-329 WebQTest-334 WebQTest-336 WebQTest-341 WebQTest-342
WebQTest-359 WebQTest-362 WebQTest-371 WebQTest-377 WebQTest-379 WebQTest-38 WebQTest-383
WebQTest-388 WebQTest-397 WebQTest-403 WebQTest-409 WebQTest-410 WebQTest-417 WebQTest-419
WebQTest-425 WebQTest-43 WebQTest-431 WebQTest-432 WebQTest-433 WebQTest-440 WebQTest-441
WebQTest-447 WebQTest-462 WebQTest-464 WebQTest-469 WebQTest-477 WebQTest-479 WebQTest-480
WebQTest-487 WebQTest-490 WebQTest-513 WebQTest-514 WebQTest-516 WebQTest-518 WebQTest-519
WebQTest-523 WebQTest-537 WebQTest-542 WebQTest-543 WebQTest-545 WebQTest-555 WebQTest-567
WebQTest-579 WebQTest-585 WebQTest-59 WebQTest-599 WebQTest-605 WebQTest-608 WebQTest-617
WebQTest-618 WebQTest-624 WebQTest-636 WebQTest-637 WebQTest-642 WebQTest-658 WebQTest-66
WebQTest-661 WebQTest-663 WebQTest-665 WebQTest-670 WebQTest-689 WebQTest-698 WebQTest-701
WebQTest-703 WebQTest-704 WebQTest-710 WebQTest-721 WebQTest-725 WebQTest-734 WebQTest-735
WebQTest-737 WebQTest-738 WebQTest-74 WebQTest-742 WebQTest-748 WebQTest-749 WebQTest-769
WebQTest-776 WebQTest-783 WebQTest-784 WebQTest-788 WebQTest-8 WebQTest-812 WebQTest-83
WebQTest-830 WebQTest-832 WebQTest-838 WebQTest-848 WebQTest-86 WebQTest-861 WebQTest-865
WebQTest-866 WebQTest-867 WebQTest-869 WebQTest-873 WebQTest-875 WebQTest-88 WebQTest-881
WebQTest-884 WebQTest-893 WebQTest-896 WebQTest-901 WebQTest-910 WebQTest-911 WebQTest-915
WebQTest-919 WebQTest-923 WebQTest-925 WebQTest-93 WebQTest-933 WebQTest-936 WebQTest-942
WebQTest-958 WebQTest-959 WebQTest-96 WebQTest-960 WebQTest-966 WebQTest-969 WebQTest-973
WebQTest-975 WebQTest-978 WebQTest-980 WebQTest-983 WebQTest-985 WebQTest-991 WebQTest-994
WebQTest-996
```

C.2 CWQ test sample ($n = 400$)

Source: results/phase4/test_cwq.json.

WebQTest-1000_0adacc5f8d85e317c77fbf2ffa83dae9 WebQTest-1000_1d3a86f839023bb35d439f56a673dbe6
WebQTest-1000_6099ea03f4fd2476605c4a513318d29c WebQTest-1000_7457bb008a1da743b19ff9ce3a5cac63
WebQTest-1002_806620bbf6ca7cf3ced5d4ff130c714d WebQTest-1002_9a330187030f30935208376dd179f40d
WebQTest-1012_131eaad9f6b360a7166467b5784470d4 WebQTest-1012_73741811f34519b29f7d19ccfd4d9553
WebQTest-1012_872253e47dd6ddaa213ff31eeda8783b WebQTest-106_8a7ae3422867fd4b2d40b06930915468
WebQTest-106_c8290c381954f40b4b920252a7df33da WebQTest-1072_92f27159a96f716c1691371e64e5a604
WebQTest-1091_61efa0b17cc53c2187422d88bf670f22 WebQTest-1120_5fd34c6d0f1cf393b2bd58d6e9690993
WebQTest-1120_c78405f6f2157645f1098f8a871f1bc2 WebQTest-1168_1591d6a0e6cef93c6863edb2eb0e198f
WebQTest-1171_a8ed8aa8e8eaf11e4b00b81b2ac2ac23 WebQTest-1178_09e61020934818b85641a22821e7a455
WebQTest-1251_cbf2f20f6caf754bc49d672ca7b150b7 WebQTest-1251_cf6cc4cc9ed790243a390f155ae72256
WebQTest-1260_018dc0c070e53555fa37f63d4b83ca87 WebQTest-1260_c4e06c3a9e4b4f10bd1ae97f1742c198
WebQTest-12_5373d9d04da65179a41e5d4669db121e WebQTest-12_7b54b31f3e5a6273f4fd2a20e565ec6d
WebQTest-12_974c964a1a2810facfeb4b1a996e878a WebQTest-1301_5714a7a5f1feec1348045800b0c0c28c
WebQTest-1320_2492cfb926882a4bf9c580ba4f7c700a WebQTest-1348_3f44e0c4fd23a8334a972ebb812e0e35
WebQTest-1348_9cd662e0ee5c2ffbbbb0dc86053b1680 WebQTest-1348_b7598df908bf8cbe941f82e1cefaec28
WebQTest-1379_8b1ff0551fb22f1d4e54d33d3656b8e8 WebQTest-1384_9440ac3bd6d8053ca4d553da408a7cb0
WebQTest-1387_e519712761951b558bb7cbf4ea39198a WebQTest-1470_888604e15ae965565efd2aa1eb615bab
WebQTest-1473_3d4ecc1165ae1c21499afb8f7b9e25a3 WebQTest-1513_5422fa602469c38418171ad342583759
WebQTest-1528_04810196a28df9032811889ca112f28b WebQTest-1528_3c737d4a4dbf3b5acb082a4a1e43d792
WebQTest-1528_505f2eda6be64caf7b48a166de833564 WebQTest-1528_6ecf5c92092ae9080307178a28b8d589
WebQTest-1528_ac9d054b50d6d55401844da4807076b2 WebQTest-1547_9a301711fdf36a2e2d6fdd7b21d65d3c
WebQTest-1560_210d8463ee35ff65448a711c4461f122 WebQTest-1560_9b121a33fc0ee97ee3a3c43ee4e02a78
WebQTest-1569_0b5333d98ef87008aa02d1fbc1554b05 WebQTest-1603_6f7bafc08148c28d022ab04b0e6a4aaf
WebQTest-1603_7ca9a8a41caa3634b454484965413fd3 WebQTest-1686_554811ebe1463287ee640a214683ea57
WebQTest-1686_80cf1e206617287917f0af7538f69ab8 WebQTest-1686_c374fe555611a2e3aba1ba9ffd739d13
WebQTest-1705_35c94dc6944d964963b44949d8ed512c WebQTest-1736_125140bfa1a60527bde6e40dce7fe54a
WebQTest-1785_23fbfc07989820c1a44f16de0e5291be WebQTest-1785_2fd3a482f02db22067954809fe7b222b
WebQTest-1785_6e1e55b0316051b2aec513f70a70b4e2 WebQTest-1785_759908f37fd43c40a532a73779dd793d
WebQTest-1785_98806527b045c387792db297c407c023 WebQTest-1785_9ecb1e0e419ee56c63da7b3da5e40ac9
WebQTest-1785_bd2684abe96767b0564cea78024983f2 WebQTest-1785_e0f5bd8c1ccde7ddfa3be8bdf3122ebc
WebQTest-1785_ee9df68984cde21c3be056caefd6613f WebQTest-1785_fdb7a181aeebb9a12bea9e01a41dd606
WebQTest-1797_194b220adab0abb76bdf49354cfdf3ff WebQTest-1797_202ded8e5df1c80ea9770154e24e5ffb
WebQTest-1797_2fb9e2823ccf35d2103fa8846d6f2ca8 WebQTest-1797_345a0887c7012fd7647ad0c6f9783e67
WebQTest-1797_5a1c66f408118c6342c4a0eb350b58de WebQTest-1797_dece4ddacc57788e31bf70edc99f1f37
WebQTest-1817_1bbd9ffd7b604510a3d6751b9f8ef796 WebQTest-1823_a8cd75f8533d96e3be293401d63fad4b
WebQTest-1840_3d474acada1d3230d94e79aad9206043 WebQTest-1923_037ebc0e903ab6eb5ef8ce44ecd3bd64
WebQTest-1923_29c81279ed9a982e12f82e764083db76 WebQTest-1923_2c5591f1dbe7402405004d7a0836dd3b
WebQTest-1923_3dac7aa955af6ee2ede38f6d42711a00 WebQTest-1923_44adab808bc1863fab0f07e6556afd67
WebQTest-1923_7084416ab9f72f1f1f8fc3ce7871ee4a WebQTest-1923_93e2fc83e55c4e0deaa4671a531ca523
WebQTest-1923_963f10fbb72072fc718bd8cc3e0ec12d WebQTest-1923_a64ef0f5ce397a5e1ef6fcd550ebfcfb
WebQTest-1923_a90c08a839ac4d0ac22ebbf436bb578b WebQTest-1923_b10bde702bb6c68ce3cdced85d5aff02
WebQTest-1923_eaeaff16b51e52669edc90501a373c61 WebQTest-212_7905d5d52ac1a17f68996d4a2245e682
WebQTest-213_024fd6ca0b4cb30927c22e93a552ae6c WebQTest-213_05bdda30187b7cfb104ec6e7d5ecec6a
WebQTest-213_3ec496c94ff4f83167e7d83479fa7082 WebQTest-213_4d64427814a804819d7bd73a4187dbe2
WebQTest-213_71621b15ae9777fca8c1becaa188a3bb WebQTest-213_7debc6a8247700de6db4ac7b0f4cc0e1
WebQTest-213_d9993d3e6f665d0c937cf7c0209ff9f8 WebQTest-302_88f3f649eed28a53b8a5479dc2c98cb1
WebQTest-361_5f8c05737061d5501b6a23a978b5589b WebQTest-389_27f6987805010183edbeba6b91a3cafd
WebQTest-389_544dcbf5d728f23ac97aa05f025a646a WebQTest-43_a29af73bf225f568206dcf810358236f
WebQTest-450_b68e8f1a3f19f80d7dfc10bf3796b71e WebQTest-484_db24dc739b7b00cdab7cf3d825975aed
WebQTest-493_3ce332b703593e71708a5604865c5758 WebQTest-517_e7d8b401e84db19b4965bb23aa43871d
WebQTest-537_4439b7c167b6ccf30c53b610934c4521 WebQTest-537_560ee86f4cb9196c8d757880602e8433
WebQTest-537_b7771109a84a9b56518460dc9e8878ff WebQTest-537_de17dd1afec3743e98d327e6b1912e99
WebQTest-537_e5da8cda32fb1aa37028f9f7f7b1d3a8 WebQTest-538_4767846f70218c1178e6fb0c4937e547
WebQTest-538_485b6c9f1bd7bd7972f3dd4fcc11b1e9 WebQTest-538_5f5da3e1d4ca7df9f19ce3fdfc5790e2
WebQTest-538_65e7de970f6844e8520d7785799ec19e WebQTest-538_d37989c422fef8a98e7289ca0144159c
WebQTest-538_fc5392eefef107aa88084b05ddd2e246 WebQTest-55_54e856d3c44e0ac4a33f0f0ebb7a67d6
WebQTest-576_495085ad9dd6274cce483df63474ca21 WebQTest-576_99c43635b648023af901dc341b3bab6e
WebQTest-61_5b52c9e057d0f0e6704f0b264e864dfc WebQTest-61_7bd6b37d01a372aa5af2a8d5ef53d827

WebQTest-61_9be8c18e016b713cbb7f7feb6badebc7 WebQTest-634_4f2c0d9eb022ed67ec2cc356d4d5113a
WebQTest-654_f4f38507bbec5d30bec54c8c8dd9a72b WebQTest-712_4e9134eaaafee4fd4023d41d4770f5eb
WebQTest-719_2883cc73d6e7cc887da948a32c5fd2bb WebQTest-719_402552b61d68fcf116111585da32583b
WebQTest-719_f1a83a32bf04983a5a5f9da914f867c8 WebQTest-743_2b39afadc21f0a2a4602121616db1f8c
WebQTest-743_4d2f11c63fa0611b0e29bc00f41bf7a9 WebQTest-743_e8f21e0f0c4375d30317d26f9b95cd91
WebQTest-759_33c0a2138ef3f4a4f61dbee8dc9714e5 WebQTest-783_b181354faec01d121c5f050e0d0da2fd
WebQTest-832_87afbf2481df60df8476b920f00c4247 WebQTest-832_c1d5c9c777c291a80c29c70ec43c45ad
WebQTest-832_c334509bb5e02cacae1ba2e80c176499 WebQTest-837_8c90d5499a627a3478313407b1404ecf
WebQTest-837_c4e06c3a9e4b4f10bd1ae97f1742c198 WebQTest-918_a6b0e48e01f77546d20952acdc67b396
WebQTest-924_6d72ea1b1b38e146c28c87fee57e7f96 WebQTest-924_ad15b96e5de6c60b06f2295638c2e471
WebQTest-965_e91a95bd503f0704b9cc40c6f2139524 WebQTest-983_1f8b6ebf20d1119eba090d8d0257bdc5
WebQTest-983_5117d407df6e21d30cd4893bcab84281 WebQTest-989_4b6636a049fa767111eef6ce82bf7054
WebQTest-989_6d0a1bf895925a752370575740368440 WebQTest-998_47e95c73ec76579a8d66b6e1607df389
WebQTrn-1001_33d9f1ae3911d4f69a9993902924107d WebQTrn-1001_6bcfac70e63efec42212159c644a593a
WebQTrn-1023_2ed287b0ef57067bd3a79f71c2fa849e WebQTrn-1023_6b6f19d9abc98d7eda3c6da9a66f1176
WebQTrn-1023_efcc9747755b72a5f927c5ccf984d9e4 WebQTrn-1053_4c8649801d197e6eb5377d292c0b6ca8
WebQTrn-105_8b3815ec57389b13892dd5abb108cc4e WebQTrn-1069_1eb6f5acf20855a1a2ef0d5b38673175
WebQTrn-1069_3c89af72801c057504e78b3a263a9b77 WebQTrn-1077_7bd6b37d01a372aa5af2a8d5ef53d827
WebQTrn-1077_d9bc2c5e35761bbc475b7900393fcc97 WebQTrn-1155_cb9805c4c3bedc59995289ea0f7dbf7f
WebQTrn-1180_67f5f717564163742ca2588b7e186fd8 WebQTrn-1180_d234333cfe0037241bacdcbbc5a74317
WebQTrn-1232_cb253aaf69993423e0691aca1ad6cafa WebQTrn-124_0782789f35ce79d56102e26e54e5b700
WebQTrn-124_609120802c32c3608fc379700186f76c WebQTrn-124_8a391bb9366c22ce0aadc00cfecf7e08
WebQTrn-124_92133c60417a74da674a0807a5a56eff WebQTrn-124_cb9e908c93b1f44b95c2119a3de0ab04
WebQTrn-125_003c6a046c47526d922683f22cf0e983 WebQTrn-1278_025fdfafd914ff922ab8144f527c06ec
WebQTrn-1278_51fc729996448b82cb36b98e70cf1498 WebQTrn-1278_d8e43a02200cfdff82052f8cc5395b27
WebQTrn-1283_623b8370871966e2e8aeb301482e1558 WebQTrn-1294_a4b2006ad5ffa1964eb8aa93149cba5a
WebQTrn-1297_a0da13a7b70a064b8cffb58007f6739d WebQTrn-1392_d372995c4cada937a6f201d316698ad5
WebQTrn-1394_75a63b2d78bab35a97e07d53149a29e0 WebQTrn-1405_32fd6bcd379a666c1b479eb00ba3dfc6
WebQTrn-1405_b5513034c35cc49fda740f1a0ae5b7b9 WebQTrn-1405_b98a27e21e904173168eb7517b123e51
WebQTrn-1405_ced20d844a472e7fafced76fae0a1c7c WebQTrn-1484_1fbd766e1b1bf978da23e8646cadf3a1
WebQTrn-1484_256d41885c1839681311ca57f4d7b6c4 WebQTrn-14_f56f0b81a5d0fd41e793f70c0b468455
WebQTrn-1521_8a0b4e1cf3027533ff8a45c91970ad01 WebQTrn-1557_25aa66737549ccec679a88f7ec71c350
WebQTrn-1557_573d4efe36c8edcac99c7247408e53c2 WebQTrn-1557_a26c076b0c074bdd9c4de45e28dfc752
WebQTrn-1560_3dba296e57fcc898668201e992da8b6b WebQTrn-1577_1948beb9c3dfa0c0861f67f8d27c503f
WebQTrn-1577_1fad167a955dd126cc7ca38ed92d28f0 WebQTrn-1577_5d3641e018eabc9011ca0d03911487d3
WebQTrn-1577_68d3456de4f00782e827bacf13846d5e WebQTrn-1577_ffdfe9623c3c4aa623a95045b61c29e9
WebQTrn-1597_8adc06ba12a3254370bdb3ff8e8820af WebQTrn-1629_5b23295775632ec566475a1ab8748962
WebQTrn-1646_5abd17edee676db9f12bbbe84edfb575 WebQTrn-1646_9dd9cd027e4fb356ec7f48576c94b9a8
WebQTrn-1665_4293d22de1b977d7bacb0d5896623115 WebQTrn-1665_5b2c4b0d5faeffa7e6ed3767a4a57c89
WebQTrn-1677_15a55e684b4c9a686e6865cfcad59905 WebQTrn-1677_22c4d321fc60b69e1d966c77d4107a74
WebQTrn-1677_2fe630fbb46b32aa9774a9417e843503 WebQTrn-1677_35504fa84f9be86a31a22c167e7c17ef
WebQTrn-1677_80066cc2617f4e1301adab9b3e951711 WebQTrn-1677_93664d8718c70b3379fbb31bf4743598
WebQTrn-1677_a7b0673d818a426f402b6a2c2573d137 WebQTrn-1677_bdfdca39c58846e51247fd78bc7683eb
WebQTrn-1677_c006ead7cde93a8aff111388eaa455c1 WebQTrn-1677_c606d81a0b9a8fe9f1a8a17239ebe8e2
WebQTrn-1679_25b39fc2f9ae169e151a869f436f594b WebQTrn-1706_97097ed7f8b2c898c800e8717998cb4c
WebQTrn-1722_c2ff19afb275737b0309f11e46a2d691 WebQTrn-1758_477d7040a0ed5bfecb155b8b3d3082fb
WebQTrn-1758_7365436f41a9104b900bd24685d394a3 WebQTrn-1762_ea65a065bc9b8f5eda6ea8c0716c84b8
WebQTrn-1770_540abec8ff3d2f4e81bfc5be9ea8e816 WebQTrn-1770_6325ee890e333f0d4fe222c15a95906f
WebQTrn-1770_6a7c160ace84e7908302f805739ad06d WebQTrn-1812_04dbed66ba17d35bf79f72e04bbcc776
WebQTrn-1812_688d69d484eeaffda36f06fbba9fe9fb WebQTrn-1812_82e069064d66663276c549f47e5e179d
WebQTrn-1812_af6e94960d6ecddfbb9e8bee3ca654f5 WebQTrn-1841_5d1cba94c816bec220b9f98e97493b0f
WebQTrn-1938_7322a2a4d46bf36b95bfab4418c9a32b WebQTrn-2006_4c7647e8d0b7435d37296e8103bcb33f
WebQTrn-2006_54c677761ff06a9b6e223ccb4c72a2a6 WebQTrn-2006_7993a79baca778d4e8e6cb2b1882bca4
WebQTrn-2006_e258f4c46fa686cf9c12150935e9211f WebQTrn-2023_139d69ce55d2d9feed41587eb4aa1b54
WebQTrn-2023_8349bb144acf1f6bb43d68bfa35e97c1 WebQTrn-2026_de4e12cff67645e8580d6a495196c768
WebQTrn-2026_de5bb2b6c6727d40089ae17f2650f97d WebQTrn-2047_01c02992dccda4886742ee8f9411d939
WebQTrn-2047_38077debd523fe1b08b3d6e13538402b WebQTrn-2047_3d5f855275265f74d81b862231578e33
WebQTrn-2057_989ef8c8e3ea409e00989ea93579d5ff WebQTrn-2069_0b65dea73fbce4f9f616eb5f69e9aa73

WebQTrn-2069_14d783de97238bd1ef6eb28fe7221306 WebQTrn-2069_c1d589c22e4bcb71dd3832d4a8a1dbe9
WebQTrn-2069_cf3638075fd9204c82c8adf9ae47925e WebQTrn-2100_faefa67fba5946fa87b438a6df0b8e63
WebQTrn-2152_3cdf60c15a8355981dd92e3c57ac2eed WebQTrn-2152_92fba37c9723caee68665ad9a5e4a468
WebQTrn-2152_c9be001aaf5f69c9067ba4b530ca0a93 WebQTrn-21_beeb032ce88fb3abfb8bf8bd2b657e62
WebQTrn-2209_7bc37f5ea0bc419b7ee5510daff240df WebQTrn-2209_beb5507aba5b006b3b2e0598989448c8
WebQTrn-2292_8c30eccd3cde88b0a0d389f34d086641 WebQTrn-2292_ea0bc3bb340865025c534c91eca19be9
WebQTrn-2311_a5f644b5210b936ed83ddcae101ef3ea WebQTrn-2311_a82b389c06081907ebdd12acd9382ab0
WebQTrn-2319_14ab2260ec77ebd5b3b8f666efc08fc3 WebQTrn-2319_16bcc3dca8a1c20229f4341ad409bb04
WebQTrn-2335_7cd339eefa681cdb907de4bdf65f8aae WebQTrn-237_f43785599ba51bf943f98f8d9bbf102a
WebQTrn-241_353fe30c00fbc6d3d872a689d17b240d WebQTrn-241_7a1f9047f5100e2853d54d69c6793f2d
WebQTrn-241_b96b735d1d754f27bc8986b614669f7b WebQTrn-241_f0ffa14797f7486fb0510a740e1cc8d6
WebQTrn-2429_4449720a6e25f3093a8065ae980f0221 WebQTrn-2478_3e09b108c3448248556d4c34acae3cf7
WebQTrn-2478_a887b1442ee30456f5de89006e1a6c72 WebQTrn-2540_1af583970c492fc3682ae363bbfca3d3
WebQTrn-2540_5e56c7691b80c044d23d2f29efc3d94f WebQTrn-2569_6b6fb0182356d92873272424688be466
WebQTrn-2569_acebb33e831a74414284fba7938e01c8 WebQTrn-2570_6f059734e774504c68f628670516e72c
WebQTrn-2570_99dba9274e8d6c826fdb25827fe613b3 WebQTrn-2570_ac52c51debd358141a23acc7ae7ec938
WebQTrn-2570_c47133a5a4595621bf114ffa8ed4c3e3 WebQTrn-2570_d63877a40200d18c7c4e39215a3c7019
WebQTrn-2570_ff6becc9074132acf2f300dbba1c1e4e WebQTrn-25_23b0132339644b875bf6a966c5b0c131
WebQTrn-25_2a2de50d3b65cc5d2c88f54e283a4b8a WebQTrn-25_3a7068fa96cd9e08dc2b1dabe458c85d
WebQTrn-25_403eec031cd613bfe527504362411ca0 WebQTrn-25_4d68c1f4906c41ef2f0ea4e416e2615b
WebQTrn-25_62939f05e216712e4700e993437a980a WebQTrn-25_d315c4815af961d22882675a61b35b81
WebQTrn-25_db9695c22f0f88f0733e1286c7fffe30 WebQTrn-25_ee5981878443f689128ca9ea8a269f29
WebQTrn-25_f65ceb2f4ef98392b0177705932740a5 WebQTrn-2615_48a6ac5681b2e67e07d84708115db614
WebQTrn-261_30a384ed1f686b2338e963e4a2a963d3 WebQTrn-261_7bc18657d5150879fc643779b6ad5e48
WebQTrn-2653_6c19bc1fe78f2c0015bfd29e5c12767c WebQTrn-2653_ea8442cb2c503dcbd06d9b7a77fed60d
WebQTrn-2664_4001558ccc529debde3a908a6f22b018 WebQTrn-2664_9c30227e437d561a395bf8370530b6e6
WebQTrn-2721_2d207ed91313bd46c4ffd81c9e26a912 WebQTrn-2748_4a22e87b9e3af2f4de74d5017c09de12
WebQTrn-2748_4d6c65004ee28d487bc3921f625106b5 WebQTrn-2748_5394f28651892e7656afeda366a6af04
WebQTrn-2754_7b7ab19b5492a3184356f9305cdd3f9f WebQTrn-2779_14a5adac3764601c1967451c79cbdb79
WebQTrn-2784_025fdfafd914ff922ab8144f527c06ec WebQTrn-2784_1ef455d28d4c19f07ec1bf5ca5bad8c3
WebQTrn-2784_699eef038aecb4af8f113ca8cd4081a0 WebQTrn-2784_abedc8acd0fdcd2e595d7ec6d58d6058
WebQTrn-2784_e35bbe9c7fc2677853b0df65b7b656f6 WebQTrn-2809_8deb69518817ab56417b04d421af85b3
WebQTrn-2815_3a3a861fe35922a25bb5991497496887 WebQTrn-2818_db09fc9949516c3b8e34002f5507514d
WebQTrn-2834_61efa0b17cc53c2187422d88bf670f22 WebQTrn-2834_d3301dedbdcb2d171a75be1631e8cec5
WebQTrn-2859_37bf80e89387cf09fca66eb7915a52b0 WebQTrn-2871_4c5dc1332d9eba8fde58e7b9561d4ae3
WebQTrn-2871_98925752e1e60abb73dc775f15ee38af WebQTrn-2904_0b2295cf692f73863c4d317fe2713132
WebQTrn-2904_13765c6b33acddb614acbd7e8d2c5ea1 WebQTrn-2904_99f2c6359f65cb116bf380f01af55d6b
WebQTrn-3034_2fd33d22c32307ac6536d5c5644a1bde WebQTrn-303_2ee96aa7464485d80d214b61773f4a5c
WebQTrn-303_770773a5150d1cdd3cfadfc25022720b WebQTrn-3057_5a7cd2677ed97e520a124ba9c2da6ad4
WebQTrn-3084_62ea63f7954eccb68d685d3695c0b84d WebQTrn-3084_ca8d129cc42cf94b681a91fb5ed94522
WebQTrn-3084_d7a8cbddd43733cfb78c3d41af704745 WebQTrn-3084_ec0835b1ad2d181fa3b4195d539d862a
WebQTrn-3100_d059b24adec4064377b957ca598769be WebQTrn-3123_5968b1cb8e15d4f7819fde8b72714100
WebQTrn-3141_9521cdd0a498eb745a7f49c67f6a999f WebQTrn-3170_8c4cd2a8dd5064dcd1e88389796138c7
WebQTrn-3249_0b9c49912f43238136954d463b417194 WebQTrn-3249_4fddfec17159cdf7caa87c1e3e1f34cd
WebQTrn-3249_6dbfa476f0510018f470c29993293e68 WebQTrn-3335_c55ccf5445d560b373944df6540f9b31
WebQTrn-3376_0619d288bbed0ca782e60c6f841a6051 WebQTrn-3376_3cfac4b90586c13b7348805674c750e1
WebQTrn-3376_ba923c7252439706e2733b02103b7ba5 WebQTrn-3400_675e2a842506fdbbcf3ccdec8f51cee3
WebQTrn-3515_76ac752ffcee1b2759600cbec55107c7 WebQTrn-3515_d1a0c35fff4a3a4dec452ce9b7793244
WebQTrn-3527_1661062ba428b2ec2a00e2f84257682f WebQTrn-3543_3f03848605c6758ff2230a955cd92d65
WebQTrn-3543_b5eabb006685da5755fdf968500b7cd4 WebQTrn-3543_ca8d129cc42cf94b681a91fb5ed94522
WebQTrn-3543_d7db4cdab1589b7c97a81236310a67f7 WebQTrn-3673_b0f2d7005d48b548c7022ad46eccfd31
WebQTrn-3744_6dcb413fa7f7fe52b9345f95bda76752 WebQTrn-3766_2458404b2726ce40f3d525263dc9f74c
WebQTrn-3769_6e0a31334a35147f868ae514f9eb9ab8 WebQTrn-412_47b2be470a658a605c3d174a78135849
WebQTrn-423_135747a75bf43cdc1e0a1cb41f49965e WebQTrn-42_021690b43db9ed877de0fa9d66751617
WebQTrn-436_f7cfe105d2830ecc77ff720612084de8 WebQTrn-452_006b1f1ed800c636a01f3888644cc1eb
WebQTrn-452_e1d6f53f49be4abf9b71b6f18c67ac13 WebQTrn-452_f79ffe93341047ed2d8a39e5bb7d9156
WebQTrn-452_f8c717f06ff656fba2fabfb354030958 WebQTrn-464_cab1228be4e8c16ee35a67b7ac63b264
WebQTrn-484_0bd50f38d8c9b7904b5a1187a904cd73 WebQTrn-484_d35194e42e8dfb2250164a3a10b93131

WebQTrn-484_dd6bb253556f96cc801f1b16e60ae1cf WebQTrn-493_590fff78c30981ef9566ad54a453d002
WebQTrn-493_cf7833e237b8cda1de396587129515c1 WebQTrn-497_3b4a316ec5790c8b3b2ec3c6aa23b29d
WebQTrn-513_5a15eae658cc54d81698d2e5e68e82c0 WebQTrn-557_12cad9d9eb020bdd9bfd11099ac0a07c
WebQTrn-567_1e6dd9ec51a40b6c0ec2105ed84574c5 WebQTrn-567_2c5591f1dbe7402405004d7a0836dd3b
WebQTrn-567_5b0f028745dcda2aac706f1a6b8a965c WebQTrn-567_693feb48c0515cd069014e7ca2846b37
WebQTrn-567_724a3b769671b1ea52a76af3a90687f4 WebQTrn-567_8e46718c3fc1361ff1c02b62a853b402
WebQTrn-567_94ad90ab85ded4cd5d0680cdd5c121aa WebQTrn-567_b63fe0fde281c871990cc523c2df1e9b
WebQTrn-567_bfcde480dde9e03c320b2d2cc1d67743 WebQTrn-568_9fa2b7684e5de3793c6eb46ac7025b64
WebQTrn-568_f4dd5e4885d4e73788e2f0e9a63d6035 WebQTrn-60_085ced56c2b8a859253d1cd5749c9a28
WebQTrn-60_1649fb7df8600ed128b0db6c53a483a2 WebQTrn-60_47c68c12c2afd9ae65f31a10982c43db
WebQTrn-60_4faaf65ae1cc1b138d41253c3fc0e493 WebQTrn-60_6b8ef173b0fe9eb52a18df50991df028
WebQTrn-60_7abf3620549d6d7aaf194ce37a34bbd8 WebQTrn-60_eae3033643523fd3a54b6c147fdfa728
WebQTrn-62_b2f9ab8521bc316c4c97cfbba4229932 WebQTrn-64_017078a590fa5382bab9986c451d8d08
WebQTrn-64_025fdfafd914ff922ab8144f527c06ec WebQTrn-64_08a3071aec88af141fc20ed22cfff0e2
WebQTrn-64_d8e43a02200cfdff82052f8cc5395b27 WebQTrn-683_7584ba13fff133af09542211fd90a33d
WebQTrn-710_c1d5c9c777c291a80c29c70ec43c45ad WebQTrn-710_e3d40457273785e46c5b71732713a5f4
WebQTrn-750_aa7f855c24369c18d0bfdb21bbf9d6d1 WebQTrn-750_bb35e9b8023fbf9c05df55b4245a4775
WebQTrn-831_c191674d835a2284701f16d70a146360 WebQTrn-831_f2d789d28c11bc5b682263b07195d208
WebQTrn-837_690725e14115ca059d1bcbfb022bbf1f WebQTrn-846_50383833393ddda52ad949cebcb1877f
WebQTrn-846_e852443b88aebb56535c3fb8390109ca WebQTrn-849_29d62c66b3b2bc4938023788455b5bc7
WebQTrn-849_82e069064d66663276c549f47e5e179d WebQTrn-849_8bb131f101e71d906d4a6dd2dbed06f7
WebQTrn-849_eba7931ddf28af4dfa20fb099196c91f WebQTrn-849_f0ffa14797f7486fb0510a740e1cc8d6
WebQTrn-849_fa1fffe7995213b6528da57ea4c8d226 WebQTrn-857_d585a4c2e04a9cd40dd5ec98e2456515
WebQTrn-886_36e0f3794f873b19127b97c24fcb3743 WebQTrn-894_022431f6dfa30cf2e715eb171e0437ec
WebQTrn-894_dc126021d39290ceeecabd0361b65c4b WebQTrn-945_121437233d5ec5331edb2a957e079759
WebQTrn-95_9cefb57084da782f843b2c57e373f0a7 WebQTrn-962_f69b9d7b30f2ef81193ae7b0a39c78c5

C.3 Development set ($n = 80$)

Source: results/phase3/dev80.json.

WebQTest-1267_9eeb2f38ee223b499967ac117f641e3b WebQTest-1356_903876a283f3ce8764412a6cfd6a2c44
WebQTest-1520_5f93548592ef4987720d1e3f3fe6b3d7 WebQTest-1554_b0412b83228508271932b11e338e48c6
WebQTest-1673_c10cd072338ca267c3ec5cd1333030f4 WebQTest-1781_455af05d577248799d41f84b34d67d8e
WebQTest-181_9dabd74992f88c62c2152b48ab1ee8cd WebQTest-1907_5abd17edee676db9f12bbbe84edfb575
WebQTest-1910_1e6d4aefe769586d47767d86ffdd20b5 WebQTest-2025_c99c8cba93f683f3e37f2e175659c925
WebQTest-2029_761df3934c8a087ee86d3ab6317d2d9d WebQTest-207_7bf178e825bb55452806d2e708f9ed50
WebQTest-410_67d6828a804cdeef38928c85cb93bb03 WebQTest-657_0818b36d353ffb39f85b4606b1230866
WebQTest-673_44a4fd4e33816608544b1091f2bd81ea WebQTest-673_5ecefd82a413a79be541a1bd16967800
WebQTest-693_dd11343b9c28a21731d2ce4bbdc2630d WebQTest-77_8176959b7600428c7cdb7aacd77fb5ea
WebQTest-851_9f924908bb36bf7d331fa1b933af9f58 WebQTrn-1
WebQTrn-1002_30f1a4f034fe9f3f6df3327d69fb669b WebQTrn-1052_b9b4dbcf721e6ef7838dcd3082761330
WebQTrn-1066 WebQTrn-1067_ed395c43c14a93f3e41c1758ec7e1b8a
WebQTrn-1214_ebbb01712f3eb1443b2895961ec14272 WebQTrn-1298 WebQTrn-144 WebQTrn-153 WebQTrn-1548
WebQTrn-1549 WebQTrn-1566_63779e80b19cde0010c566ec0b4a0ed6 WebQTrn-1589
WebQTrn-1603_169d8c870fa88566ce1f62da123323d6 WebQTrn-1719_124a834e764c099d727ab87c1f80ccee
WebQTrn-1719_96914535d2c1ae6120a090361e60c965 WebQTrn-1751 WebQTrn-2027 WebQTrn-2028
WebQTrn-2134 WebQTrn-2400 WebQTrn-2436 WebQTrn-254_01e2da60a2779c4ae4b5d1547499a4f8
WebQTrn-2584_0c8bcc717d50c92c31ff802641504b43 WebQTrn-2617
WebQTrn-262_c1b166cd48ca0675b0bb9815289851f3 WebQTrn-2721
WebQTrn-274_5b8fa743161c465927586720934bef48 WebQTrn-279_fd6e419f422eb640f1195f506522023e
WebQTrn-2825_5bf6906c26a34660ad81a9e0506d36fc WebQTrn-2840_abf44b12ba7b11ea7e71a11cb540d258
WebQTrn-3009_a453d9b2a3a3816e8ba70b1a240a4bdd WebQTrn-304_4ec9be4c828025d788bc5fb5ebca9e6e
WebQTrn-3083 WebQTrn-3085 WebQTrn-3109 WebQTrn-334 WebQTrn-3375
WebQTrn-3381_44d67b357829d662abe1c6dc637e219d WebQTrn-3457_b08f65da81574b8397d5f76ea4138c3d
WebQTrn-3460_f6e8b7149f2e422640a49beb753be4ef WebQTrn-3468 WebQTrn-3502 WebQTrn-3525

`WebQTrn-3604_fa4b1fc5cc9db1e3164c9b382fcdc4e7 WebQTrn-3617_258527684b01adb1fef0f02a9081ebe7 WebQTrn-3631 WebQTrn-3710 WebQTrn-430_aa25cef57eebcc069b1783e80e760069 WebQTrn-449 WebQTrn-528_9591187bdfee1d3adb757d74382745ce WebQTrn-546 WebQTrn-547_1f0f082cf9a15e8c58c9655386e1363f WebQTrn-559_9e3fc7126b544740b568757fd7ae1e88 WebQTrn-568 WebQTrn-569 WebQTrn-770 WebQTrn-90 dev-fake-01 dev-fake-02 smoke-20`

Appendix D

Annotation Instructions and Label Schema

Two annotation tasks in this thesis produced hand labels: the gold-quality adjudication of Section 5.2.4, and the failure census of Section 5.13. This appendix gives the instructions and the label schema both were carried out against, so that the categories the census reports can be read as definitions rather than as summary words.

D.1 Judge Instructions

The semantic-support judge of Section 5.5.5 is shown three things: the question, *one* entity the system asserted as an answer to it, and up to three knowledge-graph paths connecting the question's topic entity to that entity. It decides one thing: *do these paths express the relationship the question asks about, such that this entity is supported as an answer?* Mere connectedness is not support, and the prompt says so in those words.

The unit of judgement is therefore an *entity together with its paths*, not a claim and not a set of triples. That distinction is what the following rules are about, and it is why several of them have no analogue in the claim-level structural check of Section 4.12.1: that check asks whether an edge exists, whereas this one asks whether the edges that exist mean what the question asked. The sampling frame is built from asserted entities, so an abstention never reaches this task at all.

Eight rules constrain the judgement. Each was forced by a case met during calibration, and the guideline was fixed *before* the blind annotation began. That order is worth stating explicitly, because the alternative reading would invalidate the statistic it produced: a guideline revised in response to the disagreements it is later measured on records how well two passes were reconciled, not whether they agreed. The sequence was guideline first, then the blind pass of Section 5.5.5 — one hundred items labelled without opening the judge's output or the answer key — and $\kappa = 0.6995$ against a pre-registered bar of 0.70 is what that pass returned. The outcome is reported there rather than here.

1. **Judge the asked relationship, under the graph's vocabulary.** The test is whether the

path relations express what the question asked, as the graph is able to express it — not whether a better-designed schema would have said it more directly. A path that is merely *about* the right entities does not qualify.

2. **A mediator-bridged pair of edges is one relation.** Freebase routes many binary facts through an unnamed compound-value node, so the path renders as two hops. It is judged as the single relationship it encodes, on the same contract the tool layer uses (Section 4.2.3).

3. **Any one supporting path is enough.** Up to three are shown. If one of them expresses the asked relationship, the entity is supported; the others need not, and a weak path alongside a good one is not evidence against.

4. **Judge relations by role, not by type.** A relation that is generic in most contexts counts when it is itself the thing asked about — a nationality edge is generic filler for a question about films and is the entire answer for a question about citizenship.

5. **For conjunctions, every asked relationship must be covered by some specific path, jointly rather than in one chain.** A question imposing two constraints is supported when the paths together establish both, even if no single path establishes both at once.

6. **Judge each entity independently of answer-set completeness.** Whether the system found the *other* correct answers is recall, and it is measured by F1 elsewhere. This task asks only whether *this* assertion is grounded.

7. **Ignore superlative and temporal filters the graph cannot express.** Where a question asks for the first, largest, or most recent of something and the environment encodes no ordinals or date literals (Section 3.3.3), judge only the base relationship. Penalising the path for a filter the environment cannot represent would measure the environment, not the assertion.

8. **A path expressing only the question's identifying descriptor, rather than its actual ask, is unsupported.** "The 33rd president who led the nation during the war" identifies its subject twice and asks for it once; a path that only re-establishes the description has answered nothing.

Rule 8 names the *composition-echo* case; Section 5.5.5 draws out why it matters that the failure census arrives at the same mechanism independently.

D.2 Failure Census Label Schema

Every packet in the census receives one category, an optional subtype, and a one-sentence note. The note pinpoints where the trajectory *first* left the correct path — plan, then explored

relations, then backtracks, then verifier — rather than the last symptom. That ordering is the whole discipline of the exercise: a wrong answer usually has several things wrong with it by the end, and only the first one is a cause.

Convention for the verifier categories

The verifier's positive class is *claim is supported*. A false negative is therefore a claim wrongly *rejected*, and a false positive is a claim wrongly *accepted*. This is the standard convention and the one the census labels use throughout. It is stated explicitly because the gloss in the working reference originally paired the two names in reverse, and one Stage A label was filed against the reversed reading before the error was found. Stage D's labels followed the standard convention throughout. That one label was corrected; Appendix D.3 records it alongside the one other relabelling, which was made for an unrelated reason.

Table D.1: Failure categories and their subtypes. One category per packet.

Category	**Subtypes**	**Meaning**
decomposition_error	over_decomposition, paraphrase_drift, context_stripping, extraction_bug	the planner or drafting pipeline caused it
relation_selection	—	the scorer picked the wrong edge
composite_claim	conjunction_uncovered, no_set_intersection	a multi-constraint question handled only partially
premature_termination	budget, evaluator	stopped before reaching the answer
verifier_fn / verifier_fp	structural_not_semantic	hedged a right claim / passed a wrong one
kg_gap	date_literal, numeric_literal, temporal_qualifier, ordinal, data_error	the environment cannot express what the question needs
answer_selection	—	the correct candidate was retrieved and the drafter chose another
echo	topic, intermediate, granularity	the shared-attractor pattern of Section 5.14.5
gold_noise / ambiguous_question	see Table D.2	a defect in the gold or the question that survived Section 5.2.4
other	—	fits none of the above

Gold-defect subtypes

Seventeen gold defects survive into the census itself — 3 on WebQSP and 14 on CWQ, filed as gold_noise (7) or ambiguous_question (10) and reported as such in Section 5.14.6. They

are not subtyped there, because the census met each one incidentally while reading a failure, whereas the adjudication pass of Section 5.2.4 met roughly 148 flagged rows and made exactly this judgement at volume. Subtyping a defect class is only worth doing where the boundaries were drawn against many cases, so the adjudication pass supplies the reference list and the census rows are counted against its categories rather than given a vocabulary of their own. Table D.2 gives them with the counts they were derived from.

Table D.2: Gold-defect and question-defect subtypes, with the counts observed during adjudication across both datasets. The unit is the *flagged row*: 63 of the 148 rows the consensus pass surfaced were adjudicated defective, and these are their subtypes. That is not the unit of Table 5.2, which counts the 41 *questions* the adjudication excluded — a defective row does not always produce an exclusion, and Section 5.2.4 keeps the two apart deliberately.

Subtype	**Meaning**	**n**
gold_noise		
wrong_gold	the gold answer is factually incorrect	16
incomplete_gold	gold omits valid answers that systems correctly found	16
type_mismatch	gold answers at the wrong entity type or granularity, though the right type is in the graph	7
malformed_gold	gold is structurally corrupted, such as absurd entity counts or machine identifiers leaked into answer strings	3
ambiguous_question		
temporal	the question's time reference does not uniquely pick out gold's answer	9
container_vs_member	readable as asking about a container or about its members	3
place_vs_lineage	"where does X come from" answerable as a place or as an ancestry chain	3
malformed_question	two or more sub-questions conflated into one	2
birthplace_vs_upbringing	"where is X from" genuinely ambiguous between the two	1
variety	gold picks one defensible variety or register of an entity over another	1
place_name	a named place has competing referents and gold picks one	1
underdetermined	the question does not determine a single correct answer	1

D.3 Label Provenance and Regeneration

The census is hand labour recorded in files that scripts also write, so which file is authoritative for which field is not obvious from inspection. Table D.3 settles it.

After any relabelling, three scripts must run in order: pull_noplanner_categories.py, then synthesize_census.py, then build_thesis_numbers.py. Skipping the last leaves the thesis quoting the previous histogram.

Table D.3: Where each label lives, and what regenerating will do to it.

File	Status
labels_{ds}.csv	The hand-labelled census. dump_failure_packets.py rewrites the file but carries existing labels forward by question ID, so hand edits survive a re-run.
noplanner_categories_{ds}.csv	Generated, with no carry-forward. Labels belong in noplanner_discordant_{ds}.md, which pull_noplanner_categories.py transcribes into this CSV. An edit made directly to the CSV is overwritten on the next run.
labels_{ds}_dropped.csv	An input to the histogram rather than an archive: synthesize_census.py merges its rows into the totals. Nothing regenerates it, because it is written only at the moment a row leaves the population. If it is lost, recover it from version control.
sampling_manifest.json	Written but never read back. Its carried, to-read and dropped fields are progress counters. The thesis cites only the population and skip counts, which are stable across re-runs.

Corrections made after the first labelling pass

Three corrections were made after the initial pass — one to the working reference and two to labels — and are recorded here rather than silently absorbed.

- The false-positive and false-negative gloss in the working reference paired the two names in reverse of the standard convention. It was corrected, and the convention is now stated explicitly above. Stage D's labels had followed the standard convention throughout and none was changed on this account; one Stage A label had not, and is the second item below.
- WebQTrn-1294_a4b2006a (Stage D) moved from verifier_fn to answer_selection. This one is unrelated to the convention: the verifier had been correct, and the error was upstream, in the answerer.
- WebQTrn-1597_8adc06ba (Stage A, the planner-discordance census) moved from verifier_fn to verifier_fp, the original label having used the reversed convention. This is the only label the gloss error reached.

The histogram was regenerated afterwards. Two CWQ hedge rows moved between categories, one in each census; both stayed within hedge, so every total is unchanged.

D.4 The Gold-Label Consensus Pass

The detection principle is **independent multi-system consensus against gold**: when at least three of the five systems assert the same non-gold answer, the label is worth examining.

Independence is what makes the signal meaningful, so the voters are the five *systems* of the main matrix, spanning two knowledge sources and four retrieval paradigms; the four ablation conditions would not do, since they share the full system's scorer, drafter, and traversal, and their agreement would describe an architecture rather than a label. The pass emits one row per (question, consensus answer) pair, so rows and questions are reported as separate units throughout and question counts are always the deduplicated ones. Adjudication assigns each question one of exactly three verdicts — gold_ok, gold_wrong, ambiguous_question. Automatic triage cleared 15 of 89 rows on WebQSP and 2 of 59 on CWQ; the remainder were adjudicated by hand against the question text and, where a world fact was at issue, an external check.

Two Evidence Signatures of Label Error The most useful methodological result of this pass is that label errors come in two kinds with *opposite* evidence signatures, and separating them is what makes the adjudication reproducible rather than ad hoc.

A **world-fact error** — gold contradicts reality — requires *mixed* consensus, the parametric baseline and the graph-based systems agreeing against the label, which means two different knowledge sources agree. The San Antonio city council question is the worked specimen: four systems converge on Bexar County against a Comal County gold, and Appendix D.5 traces where that label came from. An **annotation-pipeline error** — gold contradicts the very graph the labels were generated from — has the opposite signature, *graph-only* consensus, where every graph-reading system finds the same right answer while the parametric baseline misses it for unrelated reasons. The specimen is WebQTest-55_54e856d3: "Who had a concert named Crazy Love Tour?" asks for a person, the graph-reading systems all return Michael Bublé, and gold lists his professions — the annotation query having traversed one relation too far. The distinction was forced rather than designed: adjudication scope had initially been narrowed to mixed-evidence flags only, and the Bublé case is graph-only and a genuine label error, so the narrower scope would have missed an entire class.

D.5 The Confirmed Benchmark Defects, Case by Case

What adjudication did confirm is nonetheless substantial: 57 questions across the two datasets where the benchmark, not the system, needed correcting. Two sets make that total, and they are *not* disjoint. That overlap is the whole reason the figure is 57 rather than the 58 an addition would give. There are 41 excluded before the census read anything (22 + 19), and 17 still visible in the merged census as gold_noise or ambiguous_question rows (3 WebQSP + 14 CWQ). WebQTrn-64_d8e43a02 is in both: it began as a census row, was adjudicated a type mismatch between question and gold, and was promoted to a formal exclusion mid-project. That promotion is why the census's own label file carries one dropped row. Counting it once gives $41 + 17 - 1 = 57$

distinct questions. The number is a set union rather than a sum, and it is taken as one in thesis_numbers.json so that the two component counts can move without the total drifting.

Two specimens carry the point. WebQTest-958 — "what are some famous people from el salvador" — has 116 gold entities, one of them a raw Freebase machine identifier (g.11b8058v7j) leaked into the answer strings: a "list some famous X" template ballooning gold past anything a finite answer could match under any metric. WebQTest-55_54e856d3 — "Who had a concert named Crazy Love Tour?" — asks for a person and receives gold listing the artist's professions, the annotation query having traversed one relation too far past Michael Bublé.

The Vicksburg pair is worth reading side by side, because two sibling questions share a gold string and receive opposite diagnoses. WebQTest-1797_5a1c66f4 asks who was president during the battle of Vicksburg, with gold Ulysses S. Grant — who took office six years afterwards. The president in 1863 was Lincoln. The full system answers that the claim could not be verified and its verifier flags it unsupported. The no-planner ablation asserts Grant confidently, with zero backtracks, and is scored a *hit* purely for restating a gold value that answers a different question. WebQTest-1797_dece4dda asks which government-position holder fought at Vicksburg — coherent as asked, since Grant was a serving general — and the full system genuinely fails, answering with the Confederate commander John C. Pemberton, a real combatant who held no government position. The qualifier is present in the sub-objective text and never filters the candidate frontier.

Same battle, same gold string: one question unanswerable and one a real composite_claim failure. The pipeline that hedged on the first was more epistemically correct than the one that "succeeded".

One process observation belongs on the record. WebQTrn-64_d8e43a02 was adjudicated gold_ok by the consensus pass — several systems converging on "Tupac Shakur" read as a topic echo — and the opposite way by the census reading of the same evidence: the question literally asks for an actor's name, and gold gives a character name from a different role in the same film. The second reading held, and the row was promoted to a formal exclusion. Two independent adjudication passes can legitimately disagree. Keeping a gold_noise category open in the taxonomy is what let that be resolved rather than settled by whichever ran first.

Where Bad Gold Comes From: Flattened Multi-Valued Relations The San Antonio case shows the provenance of one defect precisely enough to generalise. WebQTest-634_4f2c0d9e asks which county contains the place where the San Antonio City Council sits. Four systems, spanning parametric and graph-based knowledge, answer Bexar County. Gold says Comal. Adjudication recorded the verdict gold_wrong: San Antonio and its city council are in Bexar. But the census reading of the same case recorded something more specific, and more useful — kg_gap, with the note that this is a containment ambiguity in the source data, arising from annexed territory or from Freebase carrying multiple imprecise containment edges.

That is neither a pure label error nor a pure graph error. It is a **multi-valued containment relation flattened to a single value**, with the annotation pipeline then selecting the marginal one. A city can lie in more than one county; a benchmark whose gold field holds one answer per relation must choose; and the choice is made by a SPARQL query with no notion of which value is the principal one. Framing it this way explains why the consensus diagnostic caught it at all — the systems and the world agree on the principal value while the label holds the marginal one. It also predicts where else such defects will cluster: any relation that is genuinely one-to-many but stored as though it were one-to-one.

D.6 Verifier Error Cases

Five cases, all sharing one mechanism and all carrying the subtype structural_not_semantic: the claim is true and the answer correct, but no single triple states it in the phrasing the drafter used, so a transitively-true or differently-anchored fact is rejected and the retry loses the answer.

WebQTest-1133 resolves William Henry Smith across every backtrack round. The verifier flags the claim unsupported both times and the run hedges on what is very likely the right answer. WebQTest-38 resolves *all four* gold campaign entities in one attempt. The rejection forces a retry that narrows to two names and fails anyway. WebQTest-725 resolves both gold states, California and Nevada, then loses them over the four exploration rounds that follow and is rejected. It is the one case of the five where the depth budget blocked the retry, so no repair exploration ran. WebQTest-1348_b7598df9 resolves Houston Oilers correctly and grounded. The verifier rejects the phrasing "Peyton Manning's dad played for Houston Oilers" because the underlying triple is stated in terms of Archie Manning, and the retry recovers only the person's name. WebQTrn-568_f4dd5e48 resolves Mecklenburg County — the exact gold — through a sound two-hop chain from newspaper to city to county, is rejected, retries, resolves it again, and is rejected again.

The mechanism is one thing: the structural check demands a single edge asserting the drafted claim, and a transitively-true multi-hop fact does not present one. This is the same limitation Section 4.12.1 described by construction, now observed at census scale. Section 6.3 names the semantic-level check that would address it.

Wrongly Accepted: One Specimen and a Boundary Case Exactly one clean specimen exists in the whole census. WebQTrn-1597_8adc06ba asserts a Seth MacFarlane connection to *ThunderCats*, the verifier returns grounded, and a direct Cypher query confirms that no edge from MacFarlane to Lion-O or *ThunderCats* exists in the environment. The claim was accepted with nothing behind it.

The instructive companion is *not* a second defect. WebQTest-1620 asks when President Wilson was in office. The environment exposes inauguration events but no date literal, so the system

answers with two inauguration mentions. The asserted entities exist, the edges are real, and the structural check passed *correctly*. The answer is nonetheless wrong. The same holds for the WebQTrn-2570_d63877a4 case of Section 5.14.1: Woodrow Wilson was retrieved, the claim's endpoints genuinely connect in the graph, and the verifier was right to say so — while the answer was wrong for the question.

That contrast is the most useful point in this section. It is the same argument Section 2.2.2 makes and Section 1.3 builds on: structural grounding is a claim about edge existence, not about answer adequacy. A verifier that checks whether a claim is *in* the graph cannot, by construction, check whether it is *the answer*. Table 5.9's Tier-2 column is the measurement of that gap. These two cases are what it looks like in a single trajectory.

D.7 Relation-Selection and Navigation Cases

The Same Relation Gap Seen from Several Angles relation_selection is the largest category at 65 and carries no subtypes, because the mechanism is uniform: a wrong relation was explored, or the right one was explored and abandoned. Its interest is not in the individual cases but in the fact that several of them are the *same* gap seen from different angles.

WebQTest-1730 asks where the Paraná river flows. The system answers with the rivers it flows *into*, having explored geography.river.mouth and origin. A location-containment relation was never tried, and gold is the continent.

Three separate questions across both datasets — WebQTrn-1758_477d7040, WebQTest-1226, WebQTest-314 — all reach for government.governmental_jurisdiction.government, the organisation entity, when the question asks for a form of government and the answer lives on government.form_of_government.countries. One relation confusion, reproduced three times on unrelated questions.

The sharpest instance is a triplet on CWQ. All three questions sharing the stem WebQTrn-1770 appear, under different perturbations — _540abec8 via a Beyoncé documentary, _6325ee89 via the actor who played Etta James, _6a7c160a via the composer of a Beyoncé track — and all three resolve Beyoncé correctly, all three never once try people.person.children, the single relation that answers each of them, and all three hedge. Three independent entry points into the same resolver hitting the identical dead end is the cleanest evidence in the corpus that a relation gap is *systemic* rather than an artefact of one question's phrasing. It is a property of the scorer's ranking over that entity's relation set, not of the question.

The Evaluator Abandons a Good Relation premature_termination is small (8 cases) and splits into two mechanisms that must not be merged.

Five carry the evaluator subtype and are the *same* systemic pattern: the correct relation is explored

Table D.4: Census provenance. Stage D is the main reading over the full-matrix failures. Stage A is the separate reading of the questions where removing the planner flipped a failure into a success (Section 5.12). The single migrated row is a Stage D finding later promoted to a formal benchmark-defect exclusion (Section 5.14.6).

Source	WebQSP	CWQ
Stage D — main census (all remaining wrongs and hedges)	65	157
Stage D — row migrated to a benchmark exclusion	0	1
Stage A — planner-discordance census	21	15
Total failures analysed	**86**	**173**

with a solid score and the evaluator backtracks away from it and never returns. WebQTest-1226 (0.451), WebQTest-517_e7d8b401 (0.428), WebQTest-1635 (0.631), WebQTrn-710_e3d40457 (0.702), and WebQTest-314 (0.731) all show the identical shape across entirely unrelated domains — government form, constructed languages, sports drafts, mascots. A relation scoring 0.73 being explored and discarded is not a reasoning gap. It reads as a threshold or backtracking-policy defect, and it is one of the most directly fixable findings in the census.

The other three carry budget and are genuine exhaustion: WebQTest-1170 exhausts its backtrack budget wandering person-attribute relations after the direct edge returns nothing usable. WebQTest-1736_125140bf hits two real dead ends before the plan's second hop can run. The third, WebQTest-1630, is instructive because it belongs to two families at once: all five gold names surfaced across repeated evaluator rounds with a grounded verifier, and the run never transitioned from continue to answer before exhausting its step budget — a commitment failure of the kind Section 5.14.1 documents, expressed through the budget.

D.8 The Census Denominator: Three Overlapping Sets

The three sets overlap, so their sizes are not additive. This is worth stating because subtracting them in sequence gives the wrong answer, and the generator does not: scripts/dump_failure_packets.py skips their *union*. The full system fails on 98 WebQSP questions and 191 CWQ questions. Of the 22 and 19 adjudicated benchmark defects, only 12 and 15 were failures at all — the rest were questions the system answered correctly despite the label — and of the near-misses, 1 and 6. Those subsets are not disjoint from the Stage A reading: one WebQSP question is both an adjudicated defect and planner-discordant, and on CWQ one defect and one near-miss are. The union skipped is therefore 33 questions on WebQSP and 34 on CWQ. That leaves exactly the 65 and 157 that Stage D read, and the two totals close without remainder. A question that falls in both an exclusion set and Stage A is still analysed, through the Stage A row. Net of that re-entry and of the one migrated row, 12 WebQSP and 18 CWQ failures leave the analysis altogether. That gap is the difference between the failure counts above and the totals in the table.

D.9 Knowledge-Graph Gap Specimens

ordinal (9 cases) — superlatives such as "first", "last", "smallest", "earliest". Every instance shares one shape: the relevant relation is explored, candidates come back, and the minimum-or-maximum comparison across them simply never happens, because no component performs it. WebQTest-245, on the year of the first Miss America pageant, is the clearest illustration — the system answers 1925, then 1980, then 1929 across successive rounds, genuinely different years each time, because nothing sorts and takes the extreme.

numeric_literal (16 cases, the largest subtype) — internal identifiers such as tvrage_id, thetvdb_id, ISO alpha-3 codes, and Freebase's own located identifiers, which CWQ's templates invoke constantly and which no relation in the explored schema exposes. Several of these hedges are unusually honest: WebQTrn-849_f0ffa147, asking which country bordering Germany has ISO code CHE, names Switzerland in the answer text and then states explicitly that the code itself could not be confirmed from the available facts — a correct inference, correctly caveated, rather than a false assertion.

date_literal (4) and temporal_qualifier (4) — the environment exposes inauguration and election *events* but not the date values themselves, so a question about when an office was held returns event mentions (WebQTest-1620), and a question about who was president in a given year substitutes government.election.winner for a term-range check it cannot perform. WebQTest-749 answers "George H. W. Bush" for 1988: the winner of that November's election, who took office the following January.

data_error (6) — genuine coverage holes rather than expressiveness limits. WebQTrn-261_7bc18657 returns four consecutive rounds of explored=[] (max_score 0.0) because the image reference the question names resolves to nothing in the graph at all.

D.10 The Judge Validation Sample

One hundred judged items were drawn and written to a sheet stripped of the judge's verdicts, then labelled by the author *blind*: neither the judge's output file nor the answer key was opened until the sheet was saved. Blinding is what makes the resulting statistic a validity check rather than a measure of anchoring. A written annotation guideline was fixed during calibration and is reproduced in full in Appendix D.1; each of its eight rules was forced by a case, and the last of them — treat a path expressing only a question's identifying descriptor, rather than its actual ask, as unsupported — is the *composition-echo* case that the failure census later finds at scale (Section 5.14.5).

D.11 The Failure Taxonomy Schema

Category and Subtype Schema Ten categories carry the census, each naming a distinct architectural locus rather than a symptom. That choice is the substantive one. Surveys of hallucination classify by how the output fails: factuality against faithfulness, intrinsic against extrinsic [1]. That cut is the right one when the generator is a black box and the output is all there is to read. It is the wrong cut here, because the run record exposes which component produced the failure, and a taxonomy that discarded that would throw away the only information this system has over a black box. The categories are therefore named for where the trajectory went wrong: relation_selection (the scorer picked the wrong edge), composite_claim (a multi-constraint question handled partially), kg_gap (the environment cannot express what the question needs), decomposition_error (the planner or draft pipeline caused it), answer_selection (the correct candidate was retrieved and the drafter chose another), echo (a real, grounded entity one node away from the answer), premature_termination (the search stopped before the answer), verifier_fn and verifier_fp (a right claim rejected, a wrong one accepted), and gold_noise / ambiguous_question for the benchmark defects that escaped the exclusion pass of Section 5.2.4. An other category was kept open for mechanisms the schema did not anticipate. Section 5.14.1 reports what fell into it.

Subtypes are a second, open vocabulary that drives nothing and exists to group cases when writing. The full schema, with the worked examples that fixed each boundary, is Table D.1 in Appendix D.2.

One naming decision is recorded here because it was got wrong once and corrected. The verifier's *positive* class is "the claim is supported", so a false positive is a claim wrongly *accepted* and a false negative one wrongly *rejected*. The taxonomy's gloss originally paired those in the opposite order. It was corrected, two mislabelled rows were re-filed (Sections 5.14.1 and 5.14.3), and the merged histogram was regenerated. Section 5.14.3 states the convention once. Everywhere else this thesis says "wrongly rejected" and "wrongly accepted" in plain English, because with outputs labelled supported and unsupported a reader has no reliable way to infer which class is positive.

Population, Not Sample The census is a *population*, not a sample. Both datasets were read to completion: every wrong answer and every hedge produced by the full system, less three exclusion sets, each removed for a stated reason and each named here rather than folded into one.

The first is the adjudicated benchmark defects of Section 5.2.4 — 22 WebQSP and 19 CWQ questions where the gold label, not the system, is wrong. The second is the Stage B surface-form near-misses of Section 5.5: 1 WebQSP and 6 CWQ answers that strict string matching scores as failures and aggressive normalisation scores as hits. They are excluded because they are scoring artefacts rather than reasoning failures, and because Section 5.5 already reports them as

a measured quantity in their own right. The third is not an exclusion so much as a redirection: the questions read under Stage A, the planner-discordance census, are held out of the Stage D reading so that no question is labelled twice. They re-enter the histogram through their own row in Table D.4.

Only the first set removes questions from the analysis altogether. The second is reported elsewhere and the third is counted elsewhere, so the totals of Table D.4 are what the histogram is built over.

The three sets overlap, so their sizes are not additive. Appendix D.8 reconciles them and shows how the 259 read failures follow. Only the adjudicated benchmark defects remove questions from the analysis altogether; the other two exclusions are of questions that were never failures in the first place. The census is therefore a complete enumeration of what remained, not a sample of it, which is what licenses the category counts in this section being read as counts rather than as estimates.

An earlier plan drew a stratified $40 + 20$ sample from CWQ's failures. The census was read to completion instead, so there is no sampling-bias caveat on any number in this census. What does remain is a *composition* caveat: wrong answers and hedges are reported separately throughout and never pooled. They are different failure semantics (Section 5.5.2). Their proportions differ sharply between the two datasets, too. So a pooled percentage would describe neither.

Appendix E

Implementation Notes

Section 5.6.2 names three practices that carry more reproducibility weight than the dependency list, and promises they are recorded here as techniques rather than as incidents. Those are the first three sections below. Three further techniques follow, which govern how the agent itself is built and which the earlier three depend on. In each case the defect that forced the practice is given as evidence rather than as narrative.

E.1 Records Self-Describe

Every record RunLogger writes carries, alongside its result, the backbone description, a hash of the budget configuration, the run configuration, and the source commit. On the main matrix, the ablations and the smoke baselines — $6{,}060$ records — the run configuration also names the system, because the runner adds that key to it rather than RunConfig carrying it. The development sweep of Section 5.7.1 goes through the same logger and carries the same four stamps, but nothing there adds that key, so those 900 records identify their system by file name alone. The backbone qualification of Section 5.3 predates the logger and writes its own records, which carry the backbone description and none of the rest.

The cost is a few hundred bytes per question. The benefit is that provenance stops depending on where a file sits or what it is called: a record found in the wrong directory, or copied out of one, still states what produced it. The cache-replay anomaly of Section 5.3 is both what forced the practice and what shows its limit — the question was which model had produced a set of records, and the backbone stamp answered it, but those same records carry neither the configuration nor the commit, which is the gap the logger was written to close.

The generalisation is that a file path is not metadata. Directory layout is edited, archived and reorganised over a project's life, and every reorganisation is an opportunity to detach a result from the conditions that produced it. Anything a reader must know in order to interpret a record belongs inside the record.

E.2 Frozen Artifacts Are Committed; Regenerable Ones Are Not

The distinction is not about file size. It is about whether the artifact can be recreated.

The certified question sets, their half-splits, and every result file embody spent money and runs that cannot be repeated identically. They are committed. The graph import, the embedding matrices and the triple index are deterministic products of committed scripts run over public data, and are not committed even though rebuilding them takes hours.

Applying the rule requires honesty about the second category. An artifact is only regenerable if the script that generates it is committed *and* its inputs are still obtainable. Where either fails, the artifact is frozen regardless of how it was made, and Section 5.6.2 records the one case in this project where that judgement had to be made after the fact: a generated summary in the development-set archive that predates a fix applied to the runs it summarises.

E.3 Cache-Identity Verification

This is the technique of most methodological interest, and the one most readily reusable outside this thesis.

The problem it solves is stated easily and answered badly by ordinary testing. A refactor is made to code on the frozen experimental path. Tests pass. Have the reported numbers changed? Re-running the experiment answers the question, but a re-run that produces the same numbers is weak evidence, because it is exactly what a re-run would produce if the change had also altered which questions happened to succeed.

The stronger check exploits the response cache. Every model call is keyed by a hash over the model identifier, the sampling parameters, and the full prompt text. A code change that does not perturb the frozen path therefore issues *byte-identical prompts*, which means every call must be a cache hit. So:

> Re-run the affected questions and assert that the number of cache hits equals the number of model calls, and that both are non-zero.

A single cache miss proves that some prompt differed, and therefore that the change was not behaviour-preserving — before any output is compared. The assertion is stronger than output comparison because it tests the input to the non-deterministic component rather than the output of it, and it is cheap because a fully cached re-run costs nothing.

For this to work the cache hit must be indistinguishable from a live call inside the agent. The budget check still runs, the call counter still increments, and the original token counts are

replayed into the meter from the stored record. A cached re-run therefore reproduces the original budget snapshots exactly rather than reporting zeros, which is what makes the identity assertion meaningful. A separate counter records hits, so live and replayed runs remain distinguishable in analysis.

Anyone lifting this design should carry one defect with it. The replay restores prompt and completion tokens but not reasoning_tokens: the cache stores that field and never adds it back to the meter. The backbone here runs at reasoning effort none and the field is zero in every one of the 6,960 agent runs committed for this work, so "exactly" is literally true of these runs; under any reasoning-effort setting the replayed snapshot would understate the budget and the identity assertion would be checking a weaker claim than it appears to. Section 5.1.2 gives the reason the defect is recorded rather than repaired.

Section 5.1.2 reports the one occasion this certificate was demanded and returned a clean result, for the use_planner flag that must be a no-op when disabled; Section 4.8 specifies the cache whose behaviour makes the certificate meaningful.

E.4 Config Injection and the Partial-Edit Hazard

The agent is a graph of nodes whose signatures the framework, not the author, controls. Passing a run configuration into a node therefore means either closing over it at construction time or threading it through the framework's own injection mechanism. The first is easier and is a trap: a node that closes over a configuration value captures whatever was in scope when the graph was built, which for a sweep means the first condition's value silently serving every condition.

The practice that follows is narrow and worth stating plainly. *A configuration value must reach the code that uses it by exactly one route.* Where a value can arrive by two routes, an edit that updates one of them leaves the other in place, and the resulting run is neither the old condition nor the new one. This is the partial-edit hazard, and it is not caught by tests, because each route works in isolation.

The defence used here is an assertion at the entry of every node that receives a configuration, checking the type of what actually arrived. It converts a silent wrong-condition run into a crash on the first question. A run that dies immediately costs minutes; a run that completes under the wrong condition contaminates a table.

E.5 Routers Read, Nodes Write

The state object is mutated by nodes and inspected by routers. Allowing routers to write as well produces control flow that cannot be reconstructed from the trace, because the state a node observes was modified by the routing decision that reached it.

The discipline is one sentence: *routers may read state and must not write it; nodes may write state and must not route.* It costs a small amount of duplication — a value a router needs must be written by the node before it, under a name the router agrees on — and buys the property that the trace is a complete account of the run. Every claim in Section 5.13 about where a trajectory went wrong depends on that being true, because the census reads traces, not code.

Two categories of scratch key exist in the state for this reason. Node outputs are the record. Router inputs, prefixed with an underscore, exist so that a router can decide without recomputing what a node already knew.

E.6 Instrumentation Decides What Can Be Concluded

The verifier persists the claims it rejects. It does not persist the claims it accepts.

That decision was made early, on the reasonable ground that rejections are the interesting events: they are what the layer does, and they are few. Its consequence surfaced only at analysis time. Wrongful rejection — a true claim hedged — is recoverable from the logs at census scale, because every rejection is on record. Wrongful acceptance — a false claim passed — is not, because an accepted claim leaves no trace distinguishable from a claim that was never tested.

The thesis therefore reports one polarity of verifier error as a rate and the other from a single specimen recovered by manual trace reconstruction. Since wrongful acceptance is the polarity that bears directly on a hallucination claim, this is the limitation named first in Section 6.2.

The technique to extract from this is not a fix; the fix is obvious and is the first item of Section 6.3. It is the prior question:

> Before freezing an instrument, ask which claim it will be asked to support, and whether the records it keeps can support the negation of that claim as well as its assertion.

Logging the rejections was sufficient to describe what the layer does. It was not sufficient to bound what the layer misses, and only the second supports a claim about hallucination. The cost of logging both was negligible; the cost of not doing so was a limitation that no amount of later analysis could remove.

E.7 The Smoke-Set Validation Record

Two tuning sets appear in this thesis and they are not the same. This section uses the *smoke set*: twenty questions drawn from the training and validation splits, never from test, used once to validate the assembled framework's mechanics. The 80-question *development set* of Section 5.6.1,

Table E.1: Root causes of failure in the first end-to-end run, and the mechanism each one motivated. Every fix is described in the section indicated.

Observed failure	**Mechanism introduced**	**Section**
Backtracking was a deterministic no-op: the restored frontier produced identical scores, facts, and decisions until the budget drained	Ban list on abandoned (anchor, relation, direction) triples	Section 4.5.3
The evaluator's fact window showed the most recent thirty triples, so a large successful expansion pushed the answer out of view	Relevance-ranked fact window	Section 4.5
The relation list, capped at 80 by descending fanout, hid low-fanout precise relations behind hub relations	Cap raised to 300, plus the administrative-predicate blocklist	Section 4.2
Entities resolved while an objective was still incomplete were discarded	candidate_answers accumulated on every evaluation	Section 4.5
The frontier was truncated in insertion order, so the agent drifted away from the anchor that mattered	Frontier sorted by edge score before truncation	Section 4.4

on which the hyperparameters were swept, is a separate and larger construction, and only it carries any reported number. The smoke set was built to exercise specific machinery rather than to sample the benchmark distribution: six single-hop questions, eight two-hop compositions, three conjunctions, two whose shortest path traverses a mediator, and one about a deliberately non-existent entity whose correct answer is a refusal.

The first end-to-end run answered roughly three to four questions correctly; the final configuration answers thirteen to fourteen. That interval is not incidental tuning — it is where several of the mechanisms described above were shown to be necessary. It is also the one place in this thesis whose *evidence* is not fully archived. What survives is one record file, results/phase3/smoke20_a0.5_t0.2.jsonl, scoring thirteen correct, six hedged and one wrong at $\alpha = 0.5$ rather than the frozen 0.7. What does not is the before-state, read from a console log and never committed, so the three-to-four figure and the root causes of Table E.1 rest on that reading alone. Neither number enters any result, which is why the omission was tolerable and why the figures are given as ranges.

The first run produced no crashes, terminated cleanly on every question, and abstained rather than confabulated in almost every failure — which was the actual exit criterion for the framework's mechanics. Its sixteen or seventeen failures collapsed into five systematic causes, summarised in Table E.1. in Table E.1.

Three observations from this exercise bear directly on later chapters.

The two confabulations in the first run were caused by a retrieval defect, not a generation defect. Two questions about national currencies produced a confident "United States Dollar": the relation cap had hidden the currency_used relation, the beam surfaced triples about GNI per capita denominated in dollars, and the answerer pattern-matched. No retrieved triple supported

the answer. After the cap was raised both questions resolved correctly, at roughly a third of the token cost. These two traces are the motivating example for Section 4.10: a claim-level check against the traversed triples would have caught them regardless of the retrieval defect, which is precisely the argument for verifying before emitting.

The groundedness filter on resolutions was validated by a failure. On one question the evaluator listed nine correct countries, none of which appeared in any retrieved triple, because the beam had gone down an unrelated branch. The filter rejected all nine and the system abstained, which cost a point. The rejection also demonstrated, one layer earlier than intended, that a language model inside an agentic scaffold will assert entities it never retrieved, and that catching this requires an explicit check — nothing about the model's confidence distinguishes the nine countries it recalled from entities it actually found.

One instrumentation defect was found and removed. A counter reporting how many banned expansions had been skipped was computing the number of anchors instead, reporting a plausible-looking number unrelated to what it claimed to measure. It was corrected rather than deleted, and it motivates a standing rule adopted for the remainder of the work: a metric that has never been checked against a case where its correct value is known independently should not be trusted, and should not be analysed.

The Complete Navigation Algorithm

Algorithm E.1 states the loop.

E.8 Failure Modes of the Verification Layer

This section is analytical: it enumerates the ways the layer can be wrong that follow from its design, independently of how often they occur. Section 5.14.3 reports which of them actually fired. In the vocabulary of Section 5.14.3, a claim is *wrongly rejected* when the layer withholds support the environment would in fact provide, and *wrongly accepted* when it certifies a claim the environment does not support.

Mechanisms of Wrongful Rejection

Composite claims. The decomposer emits claims corresponding to what the draft asserts, not to every relationship the draft relies on. When a draft asserts a filtered set — "these teams were founded before 1996" — the decomposition is about the founding, and the membership relation that makes each team a candidate may never be claimed at all. Every team's only claim is then the unverifiable one, the filter drops every entity, and a draft containing four correct answers becomes a hedge. This is the Harbaugh question of Table E.2: *a composite assertion whose*

Algorithm E.1 AGR navigation (verification layer abstracted; see Section 4.10)

Require: question q, graph G, budgets B, blend α, threshold τ
Ensure: answer A with supporting triples S

$P \leftarrow \textsc{Plan}(q)$; **if** invalid **then** $P \leftarrow [\,q\,]$
$F \leftarrow \textsc{SeedAnchors}(P)$; $T \leftarrow \emptyset$; $C \leftarrow \emptyset$; $stack \leftarrow \emptyset$; $ban \leftarrow \emptyset$
while budgets permit **do**
  push $(F, depth, \sigma)$ onto $stack$ {σ from the previous pass; 0 initially}
  $R \leftarrow \bigcup_{e \in F} \textsc{GetRelations}(e)$
  $scored \leftarrow \alpha \cos(\cdot) + (1-\alpha)\mathrm{LLM}(\cdot)$ over R against the active objective
  $E \leftarrow$ top-β of $scored$ per anchor, excluding ban
  $T \leftarrow T \cup \textsc{GetNeighbors}(E)$; $F \leftarrow$ top-m new entities by score, **or** F unchanged if that set is empty {the dead-end trigger fires on the same condition and routes away before F is reused}
  $\sigma \leftarrow \max(\max_{c \in E} scored(c),\ \max_{c \in R} \cos(\cdot),\ \max_{c \in R_K} \mathrm{LLM}(\cdot))$ {signal maximum}
  $(d, done, \rho) \leftarrow \textsc{Evaluate}(\text{objective}, \textsc{TopFacts}(T))$
  $\rho \leftarrow \{x \in \rho : x \text{ appears in } T\}$; $C \leftarrow C \cup \rho$ {groundedness filter}
  if $done$ **then**
    advance P; **if** no objective remains **then** $d \leftarrow$ answer **else** $F \leftarrow \rho$
  end if
  if $|T_{\text{new}}| = 0$ **or** $\sigma < \tau$ **or** $d =$ backtrack **then**
    $ban \leftarrow ban \cup E$; $(F, depth) \leftarrow$ pop highest-scoring snapshot
  else if $d =$ answer **then**
    break
  end if
end while
$(A, S) \leftarrow \textsc{VerifyAndAnswer}(q, T, C)$ {Section 4.10}
return (A, S)

unverifiable component drags down its verifiable component. The conservative rule is defensible for a hallucination-mitigation system, and it is the layer's most expensive single behaviour on the development set.

Endpoints outside the traversal. μ is built from traversed triples only, so a claim about an entity the agent never walked to skips both structural routes regardless of what the graph contains, and is judged solely by a model against a fact window that by construction does not mention it.

A question-ranked window for a claim-level judgement. The entailment check's evidence is ranked for relevance to the question, not to the claim under test, so peripheral claims are systematically disadvantaged.

Conservative failure. Any exception in the entailment call marks all pending claims unsupported: infrastructure failure is recorded as rejection.

Name collisions in μ. The mapping keeps the first identifier seen for a name, so two distinct graph entities sharing a surface form collapse to one and adjacency is tested for the wrong node.

Mechanisms of Wrongful Acceptance

Relation-agnostic adjacency. Both structural routes ignore r and ignore direction, so any edge between the endpoints certifies any claimed relation between them: a claim that X is Y's mother is accepted on an edge recording that X is Y's child, and a claim that a film was directed by a person is accepted on an acting credit. That symmetry is the direct cost of allowing free-text relations and it is the layer's principal acceptance risk. Its scale is larger than Section 4.12.1's development-set figure invites, because neither route matches on the relation: the population exposed is every claim the two of them accept between them, which on test is somewhere in $[39, 2{,}008]$ of the $2{,}008$ accepted claims. The lower end is what the log pins down; the upper end is what the mechanism permits. Nothing measured in this thesis narrows the interval, and Section 6.3.2 is the change that would.

The mediator hop. verify_connection accepts a path through one mediator node. Because a Freebase mediator encodes a single n-ary fact the endpoints are genuinely co-participants in one fact, but which role each plays is exactly what the mediator encodes and the test discards: an actor, a film, a character, and a period are mutually "connected" through one performance node, and any claim relating any two of them is certified.

Vacuous certification of empty decompositions. A draft with no claims has no unsupported claims, so it is routed grounded. On the development set this describes 10 of 80 questions. The label is not false — an answer that asserts nothing asserts nothing unsupported — but it must not be read as a positive verification, and any statistic computed over grounded outcomes inherits this.

Entities that appear in no claim are never checked. This is the broadest mechanism listed here. Verification operates on claims, while the answer's entity list is a separate output of the same call, so an entity the drafter named but the decomposer never wrote into a claim passes through untouched on the grounded path and is deliberately retained by the permissive branch of the filter on the repair path (Section 4.13.4). Since 77 of 80 development questions took the grounded path, the layer's guarantee over emitted entities is in practice mediated entirely by decomposition recall, which is itself a language-model output subject to no check.

Grounded-but-vacuous answers. This is the boundary of what triple-level verification can do. An answer whose every claim is supported can still fail to answer the question: "the senators from New Jersey are the United States Senate members associated with New Jersey" decomposes into claims the graph genuinely supports, and is certified. No claim-level check can catch this, because the defect is not in any claim but in the relationship between the claims and the question. The anti-vacuity provision of Section 4.11 addresses it at drafting, which is mitigation rather than verification.

Structural acceptance is final. A claim accepted by adjacency is never examined semantically; the entailment check, the only component that looks at meaning, sees only what the structural

routes could not resolve.

What the Layer Is

Stated at the level of precision the preceding list requires: the layer establishes that the entity pairs a draft's claims relate are connected in the environment, and on the repair path it makes the emitted entity list a deterministic function of those verdicts rather than a second generation. It does not establish that the claimed relation is the one the edge records, that an entity absent from every claim was grounded at all, that the answer is responsive to the question, or that a claim it could not check is false. Those boundaries are properties of the design rather than defects discovered in it, and Section 5.13 reports how much of the residual error they account for.

E.9 Backbone Qualification: Four Determinism Rounds

Selection between gpt-4.1-mini-2025-04-14 and gpt-5.4-mini-2026-03-17 was made by a qualification script that runs both candidates over the same twenty development questions and reports tokens, calls, latency, reasoning tokens, errors, and a determinism probe. The probe re-runs a subset of questions and counts how often the *decision signature* — a hash of the plan, the expanded relations, the evaluator verdicts, and the answer, not of timings or scores — repeats exactly. The first round reported $3/3$ matches for 4.1-mini against $1/3$ for 5.4-mini; a second round with the cache disabled reversed it, $2/3$ against $3/3$. The reversal, not either number, is the finding.

Temperature-zero decoding on a hosted API is greedy, not deterministic: the logits are not bit-stable across calls, because floating-point reduction order depends on how a request is batched with other traffic, and a wobble of order 10^{-6} flips the argmax when the top two tokens are near-tied. A sequential agent then *amplifies* that, since one divergent token in a planner call changes a sub-objective's wording, which changes its embedding, which changes the beam. A model that is 98% stable per call can be 70–90% stable per trajectory, and with three probes per round an estimator of a rate in that band bounces between $1/3$ and $3/3$ by sampling variance alone. A tiebreaker was therefore pre-registered before rounds three and four: *if the two candidates finish within three matches of each other, take gpt-5.4-mini* on longevity grounds, the 4.1 line being deprecation-adjacent. Both rounds returned $2/3$ each, the pooled score was $9/12$ against $8/12$ — one match apart, inside the band — so the rule fired and the backbone was frozen on longevity.

E.10 Output-Contract Instrumentation Defects

Three instrumentation defects in this area are recorded rather than quietly fixed, in keeping with the same standing rule. *First*, a Python falsy-empty-list trap: the answerer substituted the

entire traversed set for the supporting triples whenever the verifier's list was empty, treating "the verifier ran and matched nothing" identically to "the verifier never ran", so a development trace recorded $1{,}927$ supporting triples for an answer that asserted nothing. That trace was read before the fix and never committed, so the figure cannot be recomputed here; the largest support count in any committed record is 133, and neither enters a result. The fix tests for None rather than for falsity. Its stated rationale was wider than the code, though: the intended three-way distinction between an absent key and an empty list is unreachable, because the state initialiser seeds supporting_triples to an empty list, so the fallback branch is dead code guarding a state the initialiser rules out.

Second, and unrepaired, a counter in the verifier's trace named n_structural counts every supported claim including those certified by entailment, so its name misdescribes it. Under the standing rule it is fenced: no table, figure or reported result derives from it, and the route-by-route counts of Section 4.12.1 come from the tool invocation log instead. The fence has one exception, named here because a rule applied silently is indistinguishable from a rule applied selectively — the third defect reads the counter once, to bound how often a duplicate could arise, which is admissible only because the direction of its error is known.

Third, a mismatch between Algorithm 1 and the code it describes. The algorithm accumulated E by set union; the implementation appends. Two claims in one draft sharing an endpoint pair — "X was born in Y" and "X died in Y", decomposed separately but resolving to the same two identifiers — therefore each append the same matching triples, so n_supporting_triples counts matches rather than distinct triples. The algorithm is corrected here to say what the code does, since the code is what produced the results. How often it bites cannot be recovered, the claim endpoints not being persisted either, but the opportunity can be bounded: a duplicate needs two claims matched by the adjacency route, and over the 908 verifier firings of the development and test sets, 378 report two or more supported claims. That 378 is an over-count of the exposure and knowably so, since the counter includes claims certified by the other two routes, neither of which appends anything. So n_supporting_triples is an *upper* bound on distinct supporting triples, and the median of 3 and mean of 4.1 inherit that.

E.11 Where Each Budget Is Enforced

Enforcement is confined to routers and the client, which is what keeps it auditable, but the mapping is not one site per budget. The model-call budget is checked inside the client before every call, so no node can bypass it, and exceeding it raises an exception that each node catches into a degraded path rather than a crash. The backtrack budget is checked in the post-evaluation router alone. Beam width and the anchor cap are not checks but slices: beam width in the explorer, the anchor cap in the explorer and again in the evaluator, where resolved entities become the next frontier. The remaining three are each checked in two places, and in every case the second

site exists because the first does not cover the verify–explore cycle. *Depth* and *wall-clock* are checked in the post-evaluation router and again in the post-verification router, since the retry edge re-enters the explorer without passing through the first, and without the second a repair would silently buy an expansion past the depth budget — Section 4.13 records the development trace that exposed exactly that. *Verification iterations* are checked in the post-verification router and again inside the verifier, which will not append a repair objective it has no budget to pursue; the two sites read the same counter through the same predicate, so neither can drift from the other.

E.12 Backtracking: Recorded Deviations

The implementation departs from what this section specifies in three places, and all three are recorded rather than repaired, under the standing rule of Section 4.9, since repairing them would change the frozen results this thesis reports. *First, $\sigma = 0$ is read as "no signal recorded".* A missing signal defaults to a value above any usable τ, which is right when no expansion has run yet; but 0.0 is falsy in Python, so a genuine signal maximum of exactly zero takes the same branch and the trigger does not fire at the one point where the formula says it most clearly should. This is the same falsy-empty trap Section 4.14 records in the answerer, found in a second place. *Second, at $\alpha \geq 1$ only the first term is computed*, the scorer taking an early return when the model term is disabled, so σ collapses to the blended maximum over expanded edges. The same early return is taken when the candidate list is empty, and there the two auxiliary maxima are not merely unrecorded but *stale*, left holding whatever the last pass wrote — and since the runtime memoises one scorer instance per α for the process, those values persist across every question in the run. Both defects have narrow practical reach, because a signal maximum of exactly zero and an empty candidate list both trip the **dead end** trigger, which is evaluated first; the masking is the order of the triggers rather than anything about the values, which is a weak reason for the behaviour to be correct. The second one does have a visible effect at $\alpha = 1$: it measures σ over a *smaller set*, the beam-selected non-banned edges rather than every candidate the anchors offered, so after a backtrack has banned the best relation σ falls where it would have held steady in the reference. That is exactly the embedding-only ablation, and Section 5.12.4 reports what it does to that condition's backtrack count instead of attributing the difference wholly to τ.

Third, the ban list is narrower than the jump it has to cover, and this is the deviation with a measurable consequence. What the backtracker bans is the *most recent* explorer pass; what it restores is the *highest-scoring* snapshot, frequently older than that pass. Everything expanded in between stays unbanned, so on exactly those backtracks the explorer can be handed back a frontier with all of its previously selected edges still available, and re-selects them. Over the main matrix, 53 explorer passes — 15 on WebQSP and 38 on CWQ — expanded an (anchor, relation) set identical to one the same question had already expanded: exactly the repetition the

ban list exists to prevent. The misalignment that produces it is visible in at least 65 of the 262 backtracks, where the restored depth is not the depth the most recent snapshot carried.[1] The unit test that should have caught this compares the first expansion set against the second, so it is structurally incapable of observing a gap that opens only when the stack is popped out of order: the test passes, and what it establishes is narrower than its name suggests.

E.13 The Development Sweep: Curve Shape and Threshold

Three readings, of which only the first is the one the sweep was run to obtain.

$\alpha = 0.7$ **is the maximum. The curve is not clean.** Hits and F1 rise from $\alpha = 0.3$ to $\alpha = 0.7$ and fall sharply at $\alpha = 1.0$. That is the expected shape: embedding similarity alone ($\alpha = 1.0$) is a weaker guide than the blend, and weighting the language-model judgement too heavily ($\alpha = 0.3$) makes the score noisy. But the wrong-answer column does not follow the hits column. It reads $7, 8, 5, 6$ across the four α values, peaking at $\alpha = 0.5$ — the condition that also has the fewest hedges of the verified rows. At $\alpha = 0.5$ the system commits more often and is wrong more often. That irregularity is reported rather than smoothed away, because a presented curve that hides a known kink is exactly what a defence finds. It is one sample of 80 questions, and the difference is three questions wide. It is noted, not interpreted.

τ **does almost nothing on this set. That is consistent with its design.** At $\alpha = 0.7$ and $\alpha = 0.5$ the two τ values produce identical hits, hedges, and wrong counts. At $\alpha = 0.3$ they differ only in backtrack count (16 against 18). At $\alpha = 1.0$, one wrong answer becomes a hedge and backtracks rise from 22 to 26. This is what the signal-maximum formulation of Section 4.5.3 predicts: the trigger fires only when the blended score, the embedding maximum, *and* the language-model maximum are all below threshold. That conjunction is rare when the language-model term is contributing. The $\alpha = 1.0$ row is where it has the most to do. It is exactly there that its effect is visible. The value $\tau = 0.20$ was frozen on the basis of a sweep in which it changed almost nothing. That is stated plainly rather than presented as a tuned choice.

Verification costs accuracy on this set. Against the frozen condition, the draft-only row scores 66 hits to 64 and F1 0.797 to 0.769. After the answer-entity filter of Section 4.13.4 was corrected — a change made on development data before any test question ran — the frozen condition scores 65 hits, 11 hedges, 4 wrong, F1 0.785. Across those two rows the layer costs one correct answer and removes no wrong ones on this set of 80. That comparison spans the correction, since the draft-only condition was never re-run after it, and is therefore the closest available reading rather than a matched pair. Section 5.12 finds the same thing at $n \approx 200$ on test data. Section 5.15.1

[1] A lower bound rather than a count. Identifying the popped snapshot exactly would need the scores, which the trace does not carry. Depth it does carry, and the depth series reconstructs exactly against the budget snapshot on all 800 records. A pop that restores an older snapshot sitting at the same depth is indistinguishable from a most-recent pop and is counted as one here.

explains why that is the expected result rather than a disappointing one.

One reproducibility note belongs with this table. Only the frozen condition was re-run after the filter correction, so Table 5.3 is the pre-correction sweep throughout — internally consistent, since all eight conditions ran under the same code. The committed summary file for that directory also predates the correction, so rescoring the archived runs yields $65/11/4$ for the frozen condition where the summary records $64/11/5$. The archived per-question records are authoritative.

E.14 Static GraphRAG: The Fanout Cap and Its Population

The radius is not the only thing that bounds this baseline. The fanout is, and it binds harder. Expansion retrieves at most 100 edges per topic entity and at most 20 per mediator hop, and the Cypher that fetches them carries no ORDER BY, so on any entity with more than 100 post-blocklist edges the 100 retrieved are an arbitrary sample of the neighbourhood rather than its best or its first. The population the cap acts on is exactly identifiable, because this baseline seeds from the dataset's annotated topic entities and expands nothing else: the $1{,}032$ topic mentions across the 800 test questions resolve to 711 distinct entities, each matching exactly one node at the resolver's exact-match stage (Section 3.2.6), so the full system necessarily seeded the same nodes and the tool log records which. Their median post-blocklist degree is 131, the ninetieth percentile $1{,}623$, the largest $136{,}742$, and 55.1% carry more edges than the cap retains. Since a question carries 1.29 topic entities on average, the question-level share is measured separately: on 72.5% of the 800 questions at least one topic entity is truncated, and on 59.0% every one of them is. For a hub topic the baseline answers from a small fraction of the available neighbourhood — on the largest, well under one percent — chosen without a stated criterion. The consequence for the hub-noise diagnosis of Section 5.9 is that the answer may never have entered the sample the reranker saw; and because the ordering is unspecified rather than seeded, this is the one place in the harness where a re-run is not guaranteed to reproduce the same retrieved set, though it did across the runs recorded here.

In fairness the same property holds of the full system's own get_neighbors, whose Cypher also carries LIMIT without ORDER BY (Appendix B.3). The two are not comparably exposed: the agent's cap is 200 rather than 100 and is applied per relation rather than per entity, so it binds on 3.3% of its $16{,}301$ neighbour calls against the baseline's 55.1% of topic entities. Section 4.2's description of the tools as parameterised Cypher templates should be read with that qualification: the parameters are fixed, the row order under a binding cap is not.

Table E.2: The complete attribution census: every development-set question whose answer differed between claim checking and its ablation, at $\alpha = 0.7$, $\tau = 0.20$. Three questions, one run per condition. "Pre-fix" is the original design in which the rewrite call regenerated the entity list. "Post-fix" is deterministic filtering. Per Section 5.5.2, the three fabricated-entity questions — the unanswerable_fake bucket, whose correct answer is a refusal — hedge in every condition and are counted as hedges rather than excluded, so the totals are over all 80. These are not the unanswerable-in-environment class of Section 3.3.3, which is a separate bucket of four whose questions do have true answers.

Question	**Draft-only**	**Verified, pre-fix**	**Verified, post-fix**
Harbaugh's teams founded before 1996 *gold:* 4 teams	4 gold teams *plus* Baltimore Ravens; scores as a hit	[] — hedge	[] — hedge
Party of Hitler and Carl Voss *gold:* Nazi Party	Nazi Party — hit	Adolf Hitler, Nazi Party; hit, but asserts the question's subject	Nazi Party — hit
Franco's university, founded 1701 *gold:* University Yale	University Yale — hit	Yale University — **wrong**	University Yale — hit
Whole run — all 80 questions; the three counts partition the set			
hits / hedges / wrong	66 / 10 / 4	64 / 11 / 5	65 / 11 / 4

E.15 The Entity-Filter Attribution Census

The classification is unambiguous. *No entailment verdict was wrong*. No repair exploration was involved — as Section 4.13 records, none ran. All three differences trace to one component: the rewrite call, failing three different ways. On Harbaugh it over-deleted, discarding four grounded-correct team names along with the unsupported one. On Hitler/Voss it laundered the question's subject into the answer list — an entity the draft never asserted. On Franco it silently respelled the graph's University Yale as the parametric Yale University, converting a defensible qualified answer into a scored error.

The finding is worth stating in one sentence, because it is the conclusion this chapter's design rests on:

> The verification logic was sound; its repair path free-generates entities from a language model, which reintroduces exactly the ungroundedness the layer exists to prevent.

Replacing generation with deterministic filtering removed two of the three regressions, exactly as predicted: Hitler/Voss and Franco left the diff.

E.16 Statistical Power of the Ablation Comparisons

One of four conditions reaches significance. That is the correct outcome to report for four components tested on half-samples. It is reported as such rather than rewritten into four confident claims.

A correction for multiple comparisons was not among the decisions fixed in advance in Section 5.1.2, so it is applied here after the fact and labelled as such. Sixteen paired tests are reported in this chapter: the eight against the baselines in Section 5.15.1 and the eight ablation comparisons above. Within the ablation family the one significant result survives correction — the planner effect on WebQSP is $p = 0.00592$ against a Bonferroni threshold of $0.05/8 = 0.00625$, and being the smallest of the eight it clears the first step of Holm's procedure at that same threshold. Nothing else in the family is near the bar under any correction. The next smallest is $p = 0.088$. The eight baseline comparisons clear $0.05/8$ as well, the largest of them at $p = 0.0035$. Correction therefore changes no claim made here. That is the only circumstance in which reporting it after the fact is worth anything.

The power limitation is arithmetic, not rhetorical. McNemar's test reads only discordant pairs, and at $n \approx 200$ with a system near 0.5–0.75 accuracy, a component that changes five questions in a hundred produces on the order of ten discordant pairs — not enough to reject at conventional levels. The planner condition cleared the bar on WebQSP because its effect is large (21 against 6), not because that comparison was better powered than the others. The halves were adopted as the pre-registered response to the benchmark's cost coming in above projection (Section 5.6). Doubling them is the straightforward remedy and is named in Section 6.2.

Three conditions are therefore reported as "no detectable effect at this sample size". That phrase is not a hedge for "no effect": for the verifier the point estimates are near zero *and* the mechanism's contribution was never expected to appear in accuracy, whereas for backtracking and the α-blend the point estimates are consistently signed and the mechanism is visibly active in the logs. Those are different epistemic situations. Collapsing them into one word would lose the distinction.

E.17 Building the Entity and Relation Indexes

Entity Embeddings Entity names were embedded with `all-MiniLM-L6-v2`, a 384-dimensional sentence encoder [2] chosen for CPU throughput at this node count. Mediator nodes were skipped: embedding a string such as `m.0y5k79m` produces noise. By Section 3.2, mediators make up almost two thirds of the graph, so skipping them is also the difference between a tractable and an intractable embedding job. Vectors were written back into Neo4j and indexed with a cosine-similarity vector index.

Relation-Vocabulary Embeddings The scoring function of Section 4.4 compares candidate relations against the current sub-objective in embedding space, so the 7,058 relation names are embedded once and held in memory rather than embedded per step.

Two details make this work. Relation names are *verbalised* before encoding: people.person.place_of_birth becomes "people person place of birth", with consecutive duplicate tokens removed. The encoder was trained on natural language. So the verbalised phrase lands near a question like "where was X born" in embedding space, where the raw dotted path does not. And relations are embedded with the *same* model as entities and sub-objectives, since vectors from different models occupy unrelated spaces and their cosine similarity is meaningless.

The resulting artefact is a 7058×384 float32 matrix with a parallel name list. Because vectors are L2-normalised at encoding time, similarity against the whole vocabulary is a single matrix–vector product.

A functional check confirms the verbalisation is doing its job. Encoding the probe "where was the person born" and ranking the vocabulary against it returns people.person.place_of_birth first at cosine 0.709 and location.location.people_born_here — the inverse relation — second at 0.675, with the remaining top-five entries plausible near-misses drawn from birth-related and fictional-character namespaces. A degenerate verbaliser would have produced an unordered top five here.

Scope: Linking and Candidate Ranking, Never Ground Truth The role of embeddings in this thesis is deliberately narrow. They resolve question phrases to graph entities, and they pre-rank relation candidates so the language-model scorer can be applied to a short list rather than to thousands of relations. They never establish that a fact holds. Every factual check — whether a triple exists, whether an entity is reachable, whether a claim is supported — is answered by symbolic graph queries. Nothing in the verification layer of Section 4.10 consults an embedding.

E.18 Bulk Import into Neo4j

Import used the offline bulk importer of Neo4j Community 5.26.28, which operates on a stopped database and is orders of magnitude faster than transactional loading at this scale. Duplicate nodes and bad relationships were configured to be skipped rather than to abort the import.

After import, a uniqueness constraint on Entity.id and a full-text index on Entity.name were created. The full-text index is what the resolver's lexical stage queries.

E.19 Candidate Widths of the Agentic Baseline

A second asymmetry is not in the baseline's favour, and it bears on how Section 5.8 should be read. Both systems call the same tools but truncate the returned candidate sets at different widths: Think-on-Graph keeps the first 40 relations per entity and the first 20 neighbours per relation, the pruning widths of the reimplemented algorithm, against AGR's 300 and 200 (Table 4.3). Because get_relations returns rows sorted by *descending* fanout (Section 4.2), a binding cut discards the low-fanout tail — precisely the material Section 4.2 reports raising AGR's own cap from 80 to 300 to stop discarding. Think-on-Graph therefore runs at half of that 80: half the cap this thesis found harmful for itself, not half its own. Measured over the committed tool logs, the 40-relation cut binds on 31.6% of the $1{,}651$ entities it expanded and the 20-neighbour cut on 32.8% of its $7{,}992$ neighbour calls, against 1 of AGR's $3{,}097$ expanded entities and 3.3% of its neighbour calls. Beam width and depth follow the original paper; the candidate widths behind them do not, and are a property of this reimplementation. The consequence is a bound on one specific claim. Section 5.8 concludes that the margin over Think-on-Graph is affordability rather than search quality, and that conclusion is safe in the direction it is stated, since a narrower candidate set cannot explain why the baseline runs *out of calls*. But the residual gap on unclipped questions is measured against a baseline searching a thinner pool, so it is a lower bound on that baseline's attainable quality rather than an estimate of it. Equalising the widths is the first item in Section 6.3.5.

E.20 Verification Layer: A Worked Example

Which university, founded in 1701, did actor James Franco attend? Gold: University Yale.

Navigation resolves James Franco as the anchor and expands along education.education.student and people.person.education. The plan's first objective — find the university founded in 1701 — is never satisfied, because the environment holds essentially no date literals (Section 3.3.3). Three evaluator-triggered backtracks follow. The run terminates on the depth budget with 18 model calls, 17 of them cache replays.

The verifier drafts: *"James Franco attended University Yale, which was founded in 1701."* Its entity list is [University Yale] — copied from the fact window, in the graph's spelling. It decomposes the draft into an attendance claim and a founding claim.

The attendance claim resolves both endpoints in μ, and its endpoint pair is in the traversed adjacency set. It is supported. Four traversed triples joining the pair are recorded as evidence. The founding claim names 1701, which no traversed triple mentions, so it cannot resolve and falls to the entailment check — which correctly reports it unsupported, because the fact is genuinely absent from the environment rather than merely unretrieved. The verifier prepares a repair objective anchored on University Yale, but the depth budget is already spent, so the router

takes give_up and the exploration never resumes. There was in any case nothing to find.

The answerer now filters. University Yale appears in a supported claim, so it is kept, verbatim. The rewrite call produces prose that asserts the attendance and hedges the founding date. Its own opinion about entities is discarded. Final answer entities: [University Yale] — a hit.

The instructive part is the counterfactual. Before deterministic filtering, this identical trace — same draft, same claims, same verdicts — ended with [Yale University]. The rewrite call, asked to restate the answer, restated it in the model's own spelling rather than the graph's. The answer was scored wrong. Nothing about the verification was different. The claim-level machinery had worked perfectly. The layer still emitted an entity string that appears nowhere in the environment. That outcome is the precise thing it exists to prevent.

E.21 The Answerer's Exception Handler

Section 4.6 reports that the answerer's third path separates budget exhaustion, which is expected, from a genuine error, which is not. What carries that distinction is the recorded exception rather than the handler: the handler catches (BudgetExhausted, Exception), whose first arm is unreachable, since the former subclasses the latter. It is the trace entry, which stores the exception itself, that tells the two apart.

Whether it does so correctly is untested by anything in this work. Across all $6{,}960$ committed runs no handler of this kind fired even once. The third path is entered on a missing draft rather than on an exception, so it needs its own check, and it gets the same answer: every one of the $3{,}690$ committed records that ran this graph carries a non-empty draft.

E.22 Structural-Check Route Frequencies

Both routes are relation-agnostic. Since r is free text with no correspondence to a Freebase predicate, matching on it would reject true claims wholesale — "X played for Y" is realised through a roster mediator carrying no predicate named for playing. The design intent is a division of labour: the structural routes provide high recall for "some supporting edge exists," and semantic precision is delegated to the entailment check. Section 4.15 examines what that division actually buys and what it lets through.

On the development set the second route was consulted five times, across five distinct questions — four per cent of the 121 claims. All five returned connected, and all five had a mediator path. In three of them there was no direct edge at all, so a mediator-blind adjacency test would have rejected them. Two were direct edges the traversal had simply never walked. Query latency across those same five calls ranged from 1.9 to 75.7 ms. With $n = 5$, that range is a record

of what happened rather than a latency distribution. The route is therefore cheap and, on this evidence, correct — but it is also rare.

Rare at test scale too. There the counts are large enough to carry weight. Across the 800 test questions the verifier decomposed 2,110 claims over 828 firings, accepted 2,008 of them and rejected 102. The second route was consulted on 100 of those claims — 4.7%, the same order as the development set's four per cent — and accepted 39. Both figures are exact: the first from the verifier's own trace entries, the second from the tool log, which records one verify_connection call per claim reaching that branch because the code issues exactly one there.

What the record cannot say is how the remaining 1,969 acceptances divide between traversed adjacency and the entailment check. A claim routed to entailment leaves no per-claim trace. The n_structural counter that Section 4.14 fences counts entailment-accepted claims alongside structural ones, so it cannot separate them either. The traversed-adjacency share is bounded above by 1,969 and below by nothing at all. That is the instrumentation gap of Section 5.15.1 in a third form. It is why this paragraph reports the share of *claims* that reached the second route, which is measured, and not the share of *acceptances* that came through the first, which is not.

E.23 Linking Misses and Gate Checks

The ten unresolved mentions of Table 3.2 are not a random residue. Every one is a bare machine identifier of the form g.123852dg appearing as a CWQ topic entity — an unnamed mediator node that the benchmark supplies where a named entity was expected. Exact matching fails because no node carries that string as a *name*. The lexical stage then returns a meaningless single-character match. The correct characterisation is that these questions are anchored on nodes with no surface form, not that resolution is unreliable.

The strongest gate check re-verifies the Python-computed reachability against the live database, confirming that nothing was lost between the CSV files and the imported graph. A random sample of 400 questions flagged reachable, drawn under a fixed seed, was re-tested with a shortest-path query in Neo4j. The pass criterion is strict — these were already known to be reachable, so anything below 100% would indicate dropped edges. **All 400 verified. The failure set is empty.** Together with the zero skipped import rows and the exact count reconciliation, the environment is certified.

E.24 Reconciling the Two Coverage Measurements

These ceilings are not the ones the validation gate reported. That difference is worth naming rather than leaving for a reader to reconcile. Table 3.3 measured the full test splits — 1,628 and 3,531 questions — and found 97.3% and 99.7% reachable. The figures above are the same

quantity measured over the 400 questions actually evaluated, and they come out at 97.0% and 99.2%. Neither corrects the other: one bounds what the environment could support across the whole benchmark, and the other bounds what any system in Section 5.7 could have scored. It is the second bound that every accuracy in this thesis is read against. The gap between them is sampling. It stays under a point on both datasets because the draw preserved the strata proportions.

E.25 The Test-Set Build That Read the Wrong Split

E.26 When the Retry Route Can Fire

The retry route requires verify, depth, *and* time budget. The depth condition was added after a development trace showed a question completing with depth 5 against a maximum of 4: the retry edge re-enters the explorer without passing through the post-evaluation router, which is where the depth check lives, so each retry silently granted one expansion beyond budget. Since Section 4.7 claims budgets are hard guarantees, the hole was closed in the router rather than documented as an exception.

That condition turns out to be decisive on the development set, where **the route never executed**. On all three development questions that produced an unsupported claim, the depth budget was already exhausted when the verifier ran, so the router took give_up in every case and no repair exploration occurred. That pattern is not a coincidence of small numbers so much as a structural correlation: the verifier is reached either because the plan completed or because a budget ran out, and all three firings came from the fourteen questions that exhausted depth, none from the sixty-six that finished with search budget in hand.

That correlation does not survive the move to test, and the route executes there. Across the 800 test questions the verifier asks for a repair on 52 — 16 on WebQSP and 36 on CWQ. Exploration actually resumes on 28 of them, 10 and 18 respectively. So the development observation was about a sample of three. Generalising it would have been wrong: the depth condition blocks the retry about half the time rather than always. What this chapter cannot answer is whether the route *helps*. The census answers that in the direction the design did not intend — Section 5.14.3 reads four of the five wrongly-rejected specimens as retries that ran and lost the answer, which is where the recall cost Section 5.15.1 attributes to this layer is actually paid. What the development set establishes is therefore narrower than it looked: that the route contributed nothing *there*, not that nothing in this thesis depends on it.

One instrumentation consequence follows, and matters when reading run records. The verifier increments verify_iters and appends the repair objective *before* the router decides, so a record showing verify_iters: 1 records the verifier's intent to repair, not an executed repair. Whether

exploration actually resumed is visible only in the node sequence of the trace.

E.27 The Resolution Filter's Head–Tail Asymmetry

The table is built from tail names only, and the verifier's is not. A traversed triple contributes its tail to this lookup and not its head, so an entity the search *departed from* — a topic entity, or any intermediate the agent has since moved past — cannot be resolved as an answer by the evaluator, however the model phrases it. The structural check of Section 4.12.1 takes the opposite convention: it collapses traversed triples into undirected endpoint pairs, so heads and tails are interchangeable there.

The asymmetry is deliberate in the evaluator and worth naming because it is not obviously so. A topic entity is exactly what an echoing model reaches for (Section 5.14.5). Refusing to resolve it is a cheap structural defence against the most common degenerate answer. The cost is that a question whose gold answer genuinely *is* a topic entity, or an intermediate the plan has passed through, cannot be answered through this path at all. Those cases exist — Section 5.14.5 counts 13 echo failures, and the filter is what prevents them from being far more numerous. But the two layers disagreeing about what counts as a reachable entity is a design seam, not a proved optimum, and no experiment in this thesis isolates it.

E.28 Why verify_triple Is Not Used

Table 4.1 lists the operations. There are five, not the four originally planned. Only four of the five are reachable from a node in the final design. Both facts are consequences of the same episode and are recorded rather than tidied away.

verify_triple was built first, as the claim checker: given a claim $\langle h, r, t \rangle$ it asked whether that relation held between those entities. It did not survive contact with generated text. Claim decomposition produces relations phrased in English — "played for", "is the capital of". Freebase predicates are neither phrased that way nor reliably one edge long. So requiring the relation to match rejected claims that were true. verify_connection was added in response. It drops the relation entirely and asks only about adjacency. This version answered the question the verifier actually needed answered, and it superseded rather than complemented its predecessor. Section 4.12.1 describes the two routes the verifier ended up with. verify_triple is not one of them.

The operation is left in the API because it is exercised by the tool integration tests and remains the natural primitive for a relation-aware check. Section 6.3 proposes exactly that check. But nothing in the running loop calls it. The table says so.

References

[1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019.

Index

Generated using Postgraduate Thesis LaTeX Template, Version 1.03. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Tuesday 8th September, 2026 at 10:24am.